



Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

SWE 4801: Software Maintenance

Summer, 2024-2025

Literature Review

AI-assisted software maintenance and refactoring

Nabila Islam

210042111

Namisa Najah Raisa

210042112

Nazifa Tasneem

210042114

Program: B.Sc in SWE

Department of CSE

Date: July 22, 2026

Contents

1	AI-powered Code Review with LLMs: Early Results	8
1.1	Introduction	8
1.1.1	Topic Background	8
1.1.2	Objective	8
1.1.3	Relevance	8
1.2	Search Strategy (Methodology)	8
1.3	Thematic Synthesis	9
1.4	Critical Evaluation	9
1.4.1	Gap Addressed	9
1.4.2	Limitations of the Paper	10
1.4.3	Debates and Source Reliability	10
1.5	Conclusion and Future Directions	10
2	A Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System	11
2.1	Introduction	11
2.1.1	Objective	11
2.1.2	Relevance	11
2.2	Search Strategy (Methodology)	11
2.2.1	Databases	11
2.2.2	Keywords	11
2.2.3	Criteria	11
2.3	Thematic Synthesis	12
2.3.1	Theme 1: Evolution of Solution Paradigms	12
2.3.2	Theme 2: Anatomy of Agentic Systems	12
2.3.3	Theme 3: Maturation of Benchmarks	12
2.3.4	Theme 4: Applications Across the Lifecycle	12
2.4	Critical Evaluation	13
2.4.1	Strengths	13
2.4.2	Gaps	13
2.4.3	Contradictions	13
2.4.4	Bias/limitations	13
2.5	Conclusion and Future Directions	13
2.5.1	Summary	13
2.5.2	Future scope	13
3	Software Maintainability Index Prediction with Source Code Metrics using Machine Learning	14
3.1	Introduction	14
3.1.1	Topic Background	14
3.1.2	Objective	14
3.1.3	Relevance	14
3.2	Search Strategy (Methodology)	14
3.2.1	Databases	14
3.2.2	Keywords	14
3.2.3	Inclusion and Exclusion Criteria	15

3.3	Thematic Synthesis	15
3.3.1	Theme 1: From Formula-Based to Data-Driven Assessment	15
3.3.2	Theme 2: Prediction Targets - Change vs. Index	15
3.3.3	Theme 3: Methodological Approaches	15
3.3.4	Theme 4: Key Findings	15
3.4	Critical Evaluation	16
3.4.1	Gaps in Research	16
3.4.2	Contradictions and Debates	16
3.4.3	Bias and Limitations of Reviewed Sources	16
3.5	Conclusion and Future Directions	16
3.5.1	Summary	16
3.5.2	Future Scope	17
4	An Empirical Study on the Potential of LLMs in Automated Software Refactoring	18
4.1	Introduction	18
4.1.1	Topic Background	18
4.1.2	Objective	18
4.1.3	Relevance	18
4.2	Search Strategy (Methodology)	18
4.3	Thematic Synthesis	19
4.3.1	Theme 1: Deep Learning-Based Refactoring (Pre-LLM Era)	19
4.3.2	Theme 2: LLMs in General Software Engineering	19
4.3.3	Theme 3: LLM-Based Refactoring	19
4.3.4	Methodological Approaches of the Primary Study	19
4.3.5	Key Findings	20
4.4	Critical Evaluation	20
4.4.1	Gaps in Research	20
4.4.2	Contradictions and Debates	20
4.4.3	Bias and Limitations	21
4.5	Conclusion and Future Directions	21
4.5.1	Summary	21
4.5.2	Future Scope	21
5	An Empirical Evaluation of Code Smell Detection in Angular Applications	22
5.1	Introduction	22
5.1.1	Topic Background	22
5.1.2	Objective	22
5.1.3	Relevance	22
5.2	Search Strategy (Methodology of the Reviewed Study)	22
5.2.1	Databases and Sources	22
5.2.2	Keywords and Boolean Operators	22
5.2.3	Inclusion and Exclusion Criteria	23
5.3	Thematic Synthesis	23
5.3.1	Theme 1: The Angular Code Smell Catalog (RQ1)	23
5.3.2	Theme 2: Community Knowledge Sources (RQ2)	23
5.3.3	Theme 3: Cross-Framework Generalization (RQ3)	23
5.3.4	Theme 4: Methodological Approaches to Detection (RQ4)	24
5.3.5	Theme 5: Key Findings on Detection Effectiveness (RQ5)	24

5.4	Critical Evaluation	24
5.4.1	Gaps in Research	24
5.4.2	Contradictions and Debates	24
5.4.3	Bias and Limitations (as acknowledged by the authors)	24
5.5	Conclusion and Future Directions	25
5.5.1	Summary	25
5.5.2	Future Scope	25
6	Patterns of Developer Adoption of LLM-Generated Code Refactoring Suggestions	26
6.1	Introduction	26
6.1.1	Topic Background	26
6.1.2	Objective	26
6.1.3	Relevance	26
6.2	Dataset and Methodology	26
6.2.1	DevGPT Dataset	26
6.2.2	Keyword-Based Filtering	26
6.2.3	Data Extraction and Mapping	26
6.2.4	Similarity Metrics	26
6.3	Findings: Adoption of Refactoring Suggestions (RQ1)	27
6.3.1	Prompt Position of Adopted Suggestions	27
6.3.2	Bimodal Similarity Distribution	27
6.3.3	Token Match Rate Insight	27
6.4	Findings: Types of Modifications (RQ2)	27
6.4.1	Refactoring Goals Observed	27
6.4.2	Five Adoption Patterns	27
6.4.3	Automation-Ready vs. Context-Dependent Tasks	27
6.5	Critical Evaluation and Conclusion	27
6.5.1	Contributions and Implications	27
6.5.2	Limitations	28
6.5.3	Future Directions	28
7	A Systematic Literature Review of Agentic AI: Definitions, Architectures, and Challenges	29
7.1	Introduction	29
7.1.1	Topic Background	29
7.1.2	Objective	29
7.1.3	Relevance	29
7.2	Search Strategy (Methodology)	29
7.2.1	Databases and Sources	29
7.2.2	Keywords	29
7.2.3	Inclusion and Exclusion Criteria	30
7.3	Thematic Synthesis	30
7.3.1	Theoretical Foundations of Agency and Agentic AI	30
7.3.2	Architectural Evolution: From Symbolic Systems to LLM-Based Agents	30
7.3.3	Proposed Taxonomy of Research Fields	30
7.3.4	Domain Applications Across the Corpus	30
7.3.5	Bibliometric Trends	30
7.4	Critical Evaluation	31

7.4.1	The Intersection Analysis: Contribution vs. Gap Mapping	31
7.4.2	The Adaptability and New-Models Research Desert	31
7.4.3	Limitations of the Review Itself	31
7.4.4	Comparison with Prior Surveys	31
7.5	Conclusion and Future Directions	31
7.5.1	Summary	31
7.5.2	Future Scope	31
8	Exploring the Use of Large Language Models for Data Extraction for Systematic Reviews in Software Engineering	32
8.1	Introduction	32
8.1.1	Topic Background	32
8.1.2	Objective	32
8.1.3	Relevance	32
8.2	Background: Task and Replicated Study	32
8.2.1	The Data Extraction Task	32
8.2.2	The Replicated SMS	32
8.3	Methodology	32
8.3.1	Extraction Setup	32
8.3.2	Scoring Procedure	33
8.4	Results	33
8.4.1	Overall Performance	33
8.4.2	Per-Question Variation	33
8.5	Discussion, Threats, and Conclusion	33
8.5.1	Interpretation and Recommended Workflow	33
8.5.2	Validity Threats	33
8.5.3	Conclusion and Future Work	33
9	CodeSense: A Static Analysis Tool for Detecting Code Smells	34
9.1	Introduction	34
9.1.1	Topic Background	34
9.1.2	Motivation	34
9.1.3	Contributions	34
9.2	Background and Related Work	34
9.2.1	Taxonomy of Code Smells	34
9.2.2	Evolution of Detection Techniques	34
9.2.3	Research Gaps	35
9.3	System Design: The CodeSense Architecture	35
9.3.1	Four-Module Pipeline	35
9.3.2	Hybrid Detection Strategy	35
9.3.3	Key Design Innovations	35
9.4	Implementation	35
9.4.1	Technology Stack	35
9.4.2	Technical Challenges	35
9.5	Performance Evaluation	36
9.5.1	Dataset and Ground Truth	36
9.5.2	Evaluation Metrics and Comparison Baselines	36
9.5.3	Results	36

9.6	Discussion and Conclusion	36
9.6.1	Interpretation of Results	36
9.6.2	Limitations	36
9.6.3	Future Work	37
10	Code Smell Detection Research Based on Pre-training and Stacking Models	38
10.1	Introduction	38
10.1.1	Topic Background	38
10.1.2	Motivation	38
10.1.3	Contributions	38
10.2	Method: The SCSmell Framework	38
10.2.1	Overview of the Pipeline	38
10.2.2	Data Collection	38
10.2.3	Textual Feature Extraction	38
10.2.4	Structural Feature Extraction	39
10.2.5	Feature Selection and Sample Balancing	39
10.2.6	The Three-Layer Stacking Model	39
10.3	Experimental Setup	39
10.3.1	Research Questions	39
10.3.2	Evaluation Metrics and Target Smells	39
10.4	Results	39
10.4.1	RQ1: Overall Detection Effectiveness	39
10.4.2	RQ2: Contribution of Combined Features	40
10.4.3	RQ3: Comparison with Traditional ML Models	40
10.4.4	RQ4: Comparison with Deep Learning Models	40
10.4.5	RQ5: Time Performance	40
10.5	Discussion and Conclusion	40
10.5.1	Threats to Validity	40
10.5.2	Conclusion	40
10.5.3	Future Work	40
11	EnseSmells: Deep ensemble and programming language models for automated code smells detection	42
11.1	Introduction	42
11.2	Search Strategy (Methodology)	42
11.3	Body Paragraphs (Thematic Synthesis)	43
11.3.1	Metric- and rule/heuristic-based detection	43
11.3.2	Classical machine learning approaches	43
11.3.3	Deep learning on source-code text	43
11.3.4	Pre-trained code/language embedding models	44
11.3.5	Methodological approaches within the reviewed literature	44
11.4	Critical Evaluation	44
11.5	Conclusion and Future Directions	45
12	Human-In-the-Loop Software Development Agents	46
12.1	Introduction	46
12.1.1	Objective	46
12.1.2	Relevance	46
12.2	Search Strategy (Methodology)	46

12.3	Body: Thematic Synthesis	47
12.3.1	Key Themes	47
12.3.2	Methodological Approaches	47
12.3.3	Key Findings	48
12.4	Critical Evaluation	48
12.4.1	Gaps in Research	48
12.4.2	Contradictions	48
12.4.3	Bias / Limitations	49
12.5	Conclusion and Future Directions	49
12.5.1	Summary	49
12.5.2	Future Scope	49
13	LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead	50
13.1	Introduction	50
13.1.1	Objective	50
13.1.2	Relevance	50
13.2	Search Strategy (Methodology)	50
13.2.1	Database	50
13.2.2	Keywords	50
13.2.3	Inclusion and Exclusion Criteria	51
13.3	Body: Thematic Synthesis	51
13.3.1	Requirements Engineering	51
13.3.2	Code Generation	51
13.3.3	Software Quality Assurance	52
13.3.4	Software Maintenance	52
13.3.5	End-to-End Software Development	52
13.3.6	Key Findings from Case Studies	52
13.4	Critical Evaluation	52
13.4.1	Gaps in Research	52
13.4.2	Contradictions and Debates	53
13.4.3	Bias/Limitations	53
13.5	Conclusion and Future Directions	53
13.5.1	Summary	53
13.5.2	Future Scope	53
14	A Causal Perspective on Measuring, Explaining and Mitigating Smells in LLM-Generated Code	54
14.1	Introduction	54
14.1.1	Objective	54
14.1.2	Relevance	54
14.2	Search Strategy (Methodology)	54
14.2.1	Databases	54
14.2.2	Keywords	55
14.2.3	Criteria	55
14.3	Body: Thematic Synthesis	55
14.3.1	Theme 1: Limitations of Correctness-Centric Evaluation	55
14.3.2	Theme 2: Empirical Evidence of Smells in LLM-Generated Code	55

14.3.3	Theme 3: Emerging Benchmarks and Metrics for Structural Quality . . .	55
14.3.4	Theme 4: Causal Inference Applied to Code Generation	56
14.3.5	Theme 5: Generation-Pipeline Factors Influencing Output Structure . .	56
14.3.6	Theme 6: Human-Centered Evaluation of AI-Assisted Code Review . .	56
14.4	Critical Evaluation	56
14.4.1	Gaps in Research	56
14.4.2	Contradictions and Debates	57
14.4.3	Bias and Limitations	57
14.5	Conclusion and Future Directions	57
14.5.1	Summary	57
14.5.2	Future Scope	57
15	FEDCSD: A FEDERATED LEARNING BASED APPROACH FOR CODE-SMELL DETECTION	59
15.1	Introduction	59
15.1.1	Objective	59
15.1.2	Relevance	59
15.2	Search Strategy (Methodology)	59
15.3	Body: Thematic Synthesis	60
15.3.1	Theme 1 - Rule- and Metric-Based Detection Tools	60
15.3.2	Theme 2 - Machine Learning-Based Detection	60
15.3.3	Theme 3 - Centralized ML's Structural Limitation	60
15.3.4	Theme 4 - Federated Learning as a Privacy-Preserving Alternative . . .	61
15.3.5	Theme 5 - Security and Privacy Threats in FL	61
15.3.6	Methodological Approaches Across the Literature	61
15.3.7	Key Findings Synthesized	61
15.4	Critical Evaluation	61
15.4.1	Gaps in Research	61
15.4.2	Contradictions and Debates	62
15.4.3	Bias, Reliability, and Limitations	62
15.5	Conclusion and Future Directions	62
15.5.1	Summary	62
15.5.2	Future Scope	63
16	Contribution	64

1 AI-powered Code Review with LLMs: Early Results

1.1 Introduction

1.1.1 Topic Background

Code review is a cornerstone of the software development lifecycle in which peers inspect proposed code changes to identify bugs, enforce coding standards, and share knowledge. Because it is labor-intensive, researchers have pursued automation of its stages for over a decade. The emergence of Large Language Models (LLMs) - neural networks trained on massive text and code corpora, such as the GPT family - has reshaped this area, offering contextual understanding and generative capabilities beyond rule-based techniques.

Key terms: **static analysis** (rule-based inspection of code without execution), **code smell** (a surface indicator of deeper design problems), **technical debt** (the future cost of expedient design choices across code, design, architecture, testing, documentation, build, and infrastructure), and **LLM-based multi-agent system** (multiple prompt-specialized LLM instances coordinating on a complex task).

The paper under review, “*AI-powered Code Review with LLMs: Early Results*” by Rasheed et al., proposes a multi-agent system of four GPT-4 agents - Code Review, Bug Report, Code Smell, and Code Optimization - coordinated through a centralized messaging component to autonomously review code, detect bugs and smells, and produce actionable optimization suggestions.

1.1.2 Objective

This review aims to (i) situate the paper within research on code review automation, (ii) synthesize the themes, methods, and findings of the prior work it builds on, and (iii) critically evaluate its contribution, limitations, and open questions.

1.1.3 Relevance

Code review consumes substantial developer effort - Thongtanunam et al. found roughly 30% of reviews suffer reviewer-assignment problems - and mainstream platforms (Gerrit, CodeFlow, Phabricator) offer minimal automation. Scientifically, the field is at an inflection point: earlier deep learning automated narrow sub-tasks, whereas LLMs promise end-to-end, explanation-rich review assistance that also educates developers, making rigorous evaluation of such systems a pressing concern.

1.2 Search Strategy (Methodology)

The literature base was built from the paper’s 26-item reference list via backward snowballing, with cited works located and verified through Google Scholar, IEEE Xplore, ACM Digital Library, SpringerLink, and arXiv. Representative Boolean search strings included: (“code review” OR “modern code review”) AND (automation OR “deep learning”); (“large language model” OR GPT) AND (“code review” OR “bug detection”); and (“multi-agent” AND LLM AND “software engineering”).

Included: peer-reviewed publications and widely cited arXiv preprints (2013–2024) on code review automation, reviewer recommendation, review-comment generation, static analysis for

review, or LLMs in software engineering; English only. **Excluded:** general code generation unrelated to review and unverifiable sources.

1.3 Thematic Synthesis

The literature is organized into five themes tracing the field's evolution from process support to generative, agentic systems.

Theme 1: Reviewer recommendation and process support. Early work targeted review logistics: RevFinder (Thongtanunam et al.) recommends reviewers from file locations of past reviews; cHRev (Zanjani et al.) ranks candidates by mined contribution history; Balachandran and Singh et al. showed static analysis integration reduces reviewer burden. This theme proved review effort is a real bottleneck but left review substance untouched.

Theme 2: Learning-based outcome prediction. Shi et al.'s DACE (CNN+LSTM) forecasts change approval; Li et al. framed acceptance as multi-instance learning over changed hunks; Hellendoorn et al. applied Transformers with attention across pull-request sections. These outputs (accept/reject probabilities) carry predictive signal but give authors little actionable guidance.

Theme 3: Generating review feedback. Gupta and Sundaresan explored deep learning for review comments; Siow et al.'s CORE retrieves relevant comments via an attentional LSTM; Tufano et al. trained Transformers to revise code to satisfy reviewer comments and later showed pre-training boosts these tasks; Li et al.'s CodeReviewer introduced review-specific pre-training. This marks the shift from predicting outcomes to participating in the review dialogue.

Theme 4: LLMs for software engineering. General-purpose LLMs and code models such as Codex triggered re-examination of SE tasks. Xu et al. systematically evaluated code LLMs' strengths and failure modes; Lu et al.'s LLaMA-Reviewer showed parameter-efficient fine-tuning can adapt an open LLM for review. Consensus: LLMs give richer, context-aware feedback than task-specific models, but evaluation methodology remains unsettled.

Theme 5: Multi-agent LLM architectures. Building on the authors' prior multi-agent work (LLM evaluation via multi-agents; CodePori for autonomous development), the reviewed paper chains four prompt-specialized GPT-4 agents through a coordination layer, each refining its predecessors' output. Preliminary results report logical, actionable feedback; detection of issues static tools missed or under-explained; refactorings aligned with accepted engineering principles; and functionality-preserving optimizations.

Methodological trajectory. The field moved from (a) mining review metadata, through (b) supervised deep learning with quantitative benchmarks, to (c) prompt-based orchestration evaluated mostly qualitatively - a notable methodological regression, since the newest agentic work, including this paper, lacks the reproducible benchmarks of earlier studies.

1.4 Critical Evaluation

1.4.1 Gap Addressed

Prior tools handled logistics, coarse predictions, or narrow feedback, while static analyzers detect only known patterns with limited explanation. No prior system combined detailed review, bug detection, smell identification, optimization, and developer education in one LLM pipeline; the multi-agent decomposition is a plausible, novel response.

1.4.2 Limitations of the Paper

No quantitative evaluation. The results are descriptive: no dataset, benchmarks, precision/recall, baselines, project counts, or inter-rater procedures; claims that static tools “missed” issues lack counts or examples.

Single-model dependency. All agents are GPT-4 prompts; no ablation isolates the multi-agent structure’s benefit over one well-prompted model.

Reproducibility. Prompts, coordination logic, and test code are undisclosed, and reliance on a proprietary, evolving commercial model undermines replication.

Unaddressed LLM failure modes. Hallucinated defects, run-to-run inconsistency, context-window limits, data-leakage risks of sending proprietary code to an external API, and multi-agent cost/latency are unexamined.

Editorial issues. Template placeholder text (a fictitious “Woodstock’22” venue) and an incomplete self-citation remain in the manuscript.

1.4.3 Debates and Source Reliability

Two open debates intersect this work: whether LLM feedback complements or duplicates static analysis (Xu et al. show code LLMs have blind spots; rule-based tools retain soundness and determinism), and whether fine-tuned open models or prompted proprietary frontier models are the better foundation - a controllability-versus-capability trade-off. The bibliography mixes top venues (ICSE, ESEC/FSE, SANER, AAI, NeurIPS) with then-unreviewed preprints, including the authors’ own prior work; self-evaluation on internally chosen cases and industrial funding (Business Finland) do not invalidate the findings but reinforce the need for independent replication.

1.5 Conclusion and Future Directions

Code review automation has evolved through five stages - process support, outcome prediction, feedback generation, LLM adaptation, and multi-agent orchestration-and Rasheed et al.’s four-agent GPT-4 pipeline sits at the frontier of the last, reportedly detecting bugs and smells, proposing optimizations, and supporting developer learning beyond static analysis. Its principal weakness is the absence of rigorous, quantitative, reproducible evaluation.

The authors’ roadmap - an empirical comparison of agent-generated reviews and documentation against manual methods, study of educational impact, and extension to the full spectrum of technical debt - targets this gap. Future work should additionally benchmark multi-agent pipelines against single-model and static-analysis baselines on public datasets; quantify false-positive rates and consistency; release prompts and code for reproducibility; examine privacy-preserving deployment (for example., locally hosted open models); and measure real-world outcomes such as review turnaround time and defect escape rates in longitudinal industrial studies.

2 A Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System

2.1 Introduction

Large Language Models (LLMs) have shifted software engineering (SE) from rule-based and template-driven automation toward systems that generate, translate, and repair code from natural-language specifications. The most recent step is the *LLM-empowered agentic system*: an autonomous architecture that couples an LLM with planning, reasoning, memory, and tool use to solve complex tasks with minimal human intervention. Two terms anchor this review: **solutions** (the methodological paradigms-prompt-based, fine-tune-based, agent-based) and **benchmarks** (standardized evaluation datasets such as HumanEval and SWE-Bench).

2.1.1 Objective

This review critically examines the survey by Guo et al., which analyzes 150+ recent papers and proposes the first taxonomy explicitly connecting benchmarks to solution paradigms. It synthesizes the survey's main themes, evaluates its methodology, and identifies gaps and limitations.

2.1.2 Relevance

LLM coding agents are entering production workflows, so the fidelity of their evaluation directly affects software reliability; meanwhile the field is fragmented and fast-moving, making a consolidating benchmark-solution map a valuable navigational tool.

2.2 Search Strategy (Methodology)

2.2.1 Databases

The survey was retrieved from **arXiv** (arXiv:2510.09721v3, cs.SE); cross-referenced primary literature was traced via **Google Scholar**, **IEEE Xplore**, the **ACM Digital Library**, and **OpenReview**, reflecting its source venues (NeurIPS, ICML, ICLR, ACL, EMNLP, ICSE, FSE, ASE, TSE, TOSEM).

2.2.2 Keywords

Representative Boolean strings: ("large language model" OR "LLM") AND "software engineering"; ("code generation" OR "program repair" OR "code translation") AND "benchmark"; ("agentic system" OR "autonomous agent") AND "code".

2.2.3 Criteria

Included: papers from 2021–2025 (emphasis 2023–2025) explicitly using LLMs for SE; works contributing novel benchmarks, methods, or agent architectures with significant empirical evaluation; peer-reviewed venues prioritized, with influential preprints admitted. *Excluded*: non-LLM approaches, LLM work outside SE, non-English publications, and opinion pieces without empirical contribution.

2.3 Thematic Synthesis

2.3.1 Theme 1: Evolution of Solution Paradigms

The literature agrees that methods have progressed along an axis of increasing autonomy. **Prompt-based** methods treat the LLM as a frozen tool: precise, verifiable instructions matter most—test-driven prompts improve correctness, while raw I/O examples degrade it; structured prompting (dependency graphs in SynFix, repository sketches in Codes) constrains generation, and interactive prompting enables clarification dialogues and conversational debugging. **Fine-tune-based** methods adapt parameters: the trend is from static code snapshots to process-oriented data (SWE-GPT), data-centric scaling that lets small models rival larger ones (SWE-Dev, SWE-Gym), and preference alignment via RL (ReST-MCTS*) or RL-free DPO-style pipelines (SelfCodeAlign). **Agent-based** methods are the frontier, examined next.

2.3.2 Theme 2: Anatomy of Agentic Systems

Agents are decomposed into four capabilities. *Planning*: multi-agent role-play (MAGIS, AgileCoder), staged/hierarchical plans, and MCTS-driven dynamic planning (LingmaAgent); notably, Agentless shows a fixed three-phase pipeline can match elaborate planning. *Reasoning & self-refinement*: generate–test–revise loops driven by execution feedback (AutoCodeRover), formal verification (Alphaverus), multi-hypothesis voting, and human-in-the-loop review; many “solved” SWE-Bench issues are in fact mis-validated. *Memory*: long-term structural memory (knowledge graphs, code-graph attention in CGM) plus short-term retrieval-based working memory, with dynamic/shared variants for multi-agent settings. *Tool augmentation*: SWE-agent’s Agent–Computer Interface outperforms raw terminal access; later work adds static analyzers and theorem provers (LLMDFA), formal verifiers, simulators, and multimodal inputs (CodeV for GUI bugs).

2.3.3 Theme 3: Maturation of Benchmarks

Benchmarks moved from *function-level* to *repository-level*, *real-world* evaluation. **Code generation**: HumanEval, MBPP, and CodeXGLUE were extended by HumanEval+ (80× more tests), multilingual/class-level suites (McEval, ClassEval, BigCodeBench), and repository-level benchmarks (DevEval, SWE-smith), with DyCodeEval addressing contamination. **Code translation**: parallel corpora (AVATAR) gave way to execution-based evaluation (CodeTransOcean, HumanEval-X), explicitly rejecting BLEU-style metrics, and then to repository-level migration suites (CRUST-Bench, RepoTransBench, AlphaTrans). **Program repair**: SWE-Bench grounded evaluation in real GitHub issues; extensions target multilinguality (Multi-SWE-bench, revealing sharp drops outside Python), multimodality (OmniGIRL), 1M-token contexts (Long-CodeBench), and security (CVE-Bench; SEC-Bench success rates of only 18–34%). **Others**: code reasoning (CRUXEval), test generation (SWT-bench, TestGenEval), refactoring, optimization, and dependency/environment setup.

2.3.4 Theme 4: Applications Across the Lifecycle

Surveyed applications span the full development lifecycle: requirements refinement and design-first planning; code generation, completion, and legacy migration; bug localization, reproduction, and end-to-end autonomous repair; performance and maintainability refactoring; and automated testing, vulnerability detection, and formally verified generation. The consistent claim is that AI support now permeates every lifecycle phase, not just code completion.

2.4 Critical Evaluation

2.4.1 Strengths

Unlike prior surveys limited to program repair or code generation, Guo et al. connect 50+ benchmarks to solution strategies across tasks, supported by a clear two-dimensional taxonomy, a unified pipeline, formal task definitions, and a continuously updated GitHub repository.

2.4.2 Gaps

The survey offers no quantitative meta-analysis-benchmark scores are catalogued but never aggregated, so the benchmark–solution “connection” remains descriptive. Requirements engineering, documentation, and human factors (productivity, trust, deskilling) receive only incidental treatment, and the acknowledged domain-adaptation gap (COBOL, embedded, HPC) is not quantified.

2.4.3 Contradictions

Live debates include: how much agency is needed (Agentless vs. multi-agent frameworks ; benchmark validity (mis-validated SWE-Bench solutions); the limited benefit of execution-trace SFT, challenging a common assumption; and iterative feedback helping in dialogue but failing as raw I/O examples.

2.4.4 Bias/limitations

The survey is an unreviewed arXiv preprint with visible editorial lapses: Sections VII and VIII duplicate the challenges material nearly verbatim, and paper counts conflict (“150+” vs. “140”). Some strong claims (for example., most SWE-Bench solutions would fail production review) are uncited, coverage restricted to 2023–2025 top venues risks selection bias, and rapid field evolution threatens staleness. These issues counsel treating the synthesis as a map rather than settled evidence.

2.5 Conclusion and Future Directions

2.5.1 Summary

The survey maps a field evolving along two coupled trajectories: solutions moving from prompting through fine-tuning to autonomous agents built on planning, reasoning, memory, and tools; and benchmarks moving from synthetic function-level tests to repository-level, multilingual, security-aware evaluation grounded in real artifacts. Its central insight—that progress is bottlenecked less by model capability than by evaluation fidelity—is well supported.

2.5.2 Future scope

Building on the identified gaps, future work should (i) aggregate benchmark results into quantitative meta-analyses; (ii) build lifecycle-aware evaluation environments scoring security, performance, maintainability, and cost alongside correctness; (iii) develop hierarchical, graph-structured code representations and tiered memory; (iv) advance multi-agent coordination and continuous learning against knowledge drift; (v) create benchmarks for underrepresented languages and domains; and (vi) establish provenance, audit, and governance mechanisms for responsible deployment.

3 Software Maintainability Index Prediction with Source Code Metrics using Machine Learning

3.1 Introduction

3.1.1 Topic Background

Software Maintainability Prediction (SMP) is the process of estimating how easily a software system can be analyzed, modified, or enhanced throughout its lifecycle. It is a fundamental dimension of software quality, informing long-term productivity, maintenance cost, and reliability. Key terms include the *Maintainability Index (MI)*, a composite numerical score of code maintainability; *source code metrics* such as *comment density* (proportion of comments), *cyclomatic complexity* (number of independent execution paths), *Halstead volume* (program size based on operator/operand counts), and *cohesion* (relatedness of functionality within a module); and the *M5P Model Tree*, a hybrid algorithm combining decision-tree partitioning with linear regression models at leaf nodes.

3.1.2 Objective

This review synthesizes the literature surrounding Birant's (2026) study, which proposes an M5P Model Tree-based regression model for predicting the MI from four source code metrics, achieving a correlation coefficient (R) of 0.8892-a 9.23% improvement over competing methods. The review situates the paper within prior SMP research, evaluates its methodological rigor, and identifies gaps for future work.

3.1.3 Relevance

As software systems grow in size and complexity, accurate maintainability assessment enables early identification of maintenance risks, prioritization of refactoring, resource planning, and better design decisions. Traditional formula-based MI computation with fixed coefficients fails to capture nonlinear inter-metric relationships, motivating adaptive, data-driven machine learning (ML) approaches.

3.2 Search Strategy (Methodology)

3.2.1 Databases

The literature underpinning this review was drawn from sources indexed in Google Scholar, IEEE Xplore, SpringerLink, ScienceDirect (Elsevier), MDPI, and Kaggle (dataset repository), reflecting the venues of the reviewed paper's reference list.

3.2.2 Keywords

Representative search strings included: "software maintainability" AND "machine learning", "maintainability index" AND "prediction", "source code metrics" AND ("regression" OR "classification"), and "M5P model tree".

3.2.3 Inclusion and Exclusion Criteria

Inclusion: peer-reviewed journal or conference papers published 2019–2026; empirical studies applying ML to maintainability change or index prediction; studies reporting quantitative performance measures. **Exclusion:** non-English publications, purely theoretical work without empirical validation, and studies on unrelated quality attributes (for example., defect prediction alone).

3.3 Thematic Synthesis

3.3.1 Theme 1: From Formula-Based to Data-Driven Assessment

The literature broadly agrees that fixed-coefficient MI formulas provide only a coarse estimate of quality because metric relationships vary across projects and languages. Consequently, research has shifted toward ML models that learn patterns directly from empirical data. Systematic reviews confirm sustained growth in ML-based SMP research.

3.3.2 Theme 2: Prediction Targets - Change vs. Index

Two research directions dominate: (i) *Maintainability Change Prediction*, estimating the number or likelihood of future code changes, and (ii) *Maintainability Index Prediction*, forecasting MI values directly from code characteristics. Birant’s study belongs to the second stream. A related sub-theme is the interpretation of numerical MI as qualitative levels (Low/Medium/High), with differing thresholds across studies—for example., color-coded bands (red 0–9, yellow 10–19, green 20–100), a binary cut-off at $MI > 20$, and low (<65)/medium (65–85)/high (>85) bands.

3.3.3 Theme 3: Methodological Approaches

The field employs a wide algorithmic spectrum:

- **Classical supervised learners:** Decision Trees, SVM, Neural Networks, Logistic Regression, KNN, and Naive Bayes.
- **Ensemble methods:** Random Forest, Extra Trees, AdaBoost, Gradient Boosting, XGBoost, LightGBM, and CatBoost.
- **Deep learning:** CNNs and LSTMs for capturing complex metric interactions.
- **Hybrid/optimized models:** metaheuristic-tuned neural networks (Chimp, Harris Hawks, Manta Ray Foraging optimization).

Tasks are formulated either as classification (predicting maintainability levels) or regression (predicting continuous values). Evaluation measures differ accordingly: Accuracy, Precision, Recall, F-measure, and AUC for classification; MAE, RMSE, MAPE, and MMRE for regression. Birant uses WEKA with 10-fold cross-validation and five regression indicators (R, MAE, RAE, RMSE, RRSE), plus non-parametric significance testing (Quade test, $p = 0.00126$; Friedman Aligned Ranks test, $p = 0.01412$).

3.3.4 Theme 4: Key Findings

Birant’s M5P model outperformed eight baselines (Linear Regression, NN, KNN, KStar, RF, LWL, Random Tree, REPTree) on all five measures, achieving $R = 0.8892$, $MAE = 3.1567$, and $RMSE = 3.9640$; the nearest competitors were NN ($R = 0.8782$) and RF ($R = 0.8729$). Fold-wise MAE remained stable (2.75–3.52), indicating consistency. Correlation analysis identified

cyclomatic complexity as the dominant negative driver of maintainability ($r = -0.759$), with Halstead volume weakly negative ($r = -0.237$) and comment density ($r = 0.281$) and cohesion ($r = 0.246$) positively associated. Sensitivity analysis showed performance degrades as the *MinNumInstances* leaf-size parameter grows (R falling from 0.8892 to 0.7582 at 350), confirming that finer partitions capture subtler patterns. These results align with prior evidence of M5P's strength in other domains, where it surpassed neural networks, support vector regression, and tree-based competitors.

3.4 Critical Evaluation

3.4.1 Gaps in Research

- **Limited feature set:** Only four metrics are used; widely studied object-oriented metrics (for example., coupling, inheritance depth, CK metrics) are omitted, though prior work shows feature selection materially affects results.
- **Single dataset:** Evaluation relies on one clean Kaggle dataset drawn from Java-centric open-source projects (Apache Commons, Hadoop, Kafka, Lucene, Spring Boot), limiting cross-language and industrial generalizability.
- **Circularity risk:** Since MI is conventionally computed from metrics resembling the model inputs (for example., Halstead volume, cyclomatic complexity), the prediction target may partially encode the features, inflating apparent accuracy—an issue the paper does not address.
- **No comparison with deep learning:** Baselines exclude CNN/LSTM approaches shown effective elsewhere, and modern boosting methods (XGBoost, CatBoost) are absent from the comparison table.

3.4.2 Contradictions and Debates

The field lacks consensus on (i) MI level thresholds—schemes are mutually inconsistent, complicating cross-study comparison; (ii) task formulation (classification vs. regression); and (iii) the best algorithm family—ensembles, deep networks, and model trees each claim superiority on different datasets, suggesting performance is dataset-dependent.

3.4.3 Bias and Limitations of Reviewed Sources

The paper's dataset is described as having “no missing values or outliers,” which is unrepresentative of real-world code repositories and may overstate robustness claims. The 9.23% improvement is computed against baselines on a single dataset without external validation. Several cited comparative M5P successes come from non-software domains (rainfall, air quality, IoT), weakening transferability of that evidence. More broadly, many SMP studies rely on small or legacy datasets (for example., UIMS/QUES), raising reproducibility and imbalance concerns.

3.5 Conclusion and Future Directions

3.5.1 Summary

The literature demonstrates a clear evolution from static, formula-based maintainability indices to adaptive ML models. Within this landscape, Birant's M5P Model Tree approach offers a compelling balance of accuracy ($R = 0.8892$), interpretability (explicit tree structure with linear leaf models), robustness (pruning and smoothing), and computational efficiency, with

statistically significant gains over eight classical baselines. Cyclomatic complexity emerges as the most influential maintainability factor across analyses.

3.5.2 Future Scope

Future research should (i) validate M5P on multi-language, industrial, and noisy datasets; (ii) enrich the feature space with object-oriented and process metrics combined with feature selection; (iii) benchmark against deep learning and gradient boosting families under identical protocols; (iv) address potential feature–target circularity by predicting externally validated maintainability outcomes (for example., actual maintenance effort); and (v) build the practitioner-facing tooling envisioned by the author-quality dashboards and refactoring-support interfaces-to translate predictive accuracy into engineering practice.

4 An Empirical Study on the Potential of LLMs in Automated Software Refactoring

4.1 Introduction

4.1.1 Topic Background

Software refactoring is the process of restructuring existing source code without changing its external behavior, with the goal of improving readability, maintainability, and reusability. Classic tool support-code smell detectors, refactoring recommenders, and refactoring engines such as IntelliJ IDEA and JDeodorant-automates the *execution* of refactorings, but developers must still decide *where* to refactor, *what type* of refactoring to apply, and with *which parameters*. Large language models (LLMs) such as GPT-4 and Gemini, having demonstrated strong performance in code generation, fault localization, and program repair, are natural candidates for automating these decisions. Key terms include: *refactoring opportunity identification* (locating poorly designed code), *refactoring solution recommendation* (proposing the concrete change), and *unsafe refactoring* (an LLM output that alters program semantics or introduces syntax errors, i.e., a form of hallucination).

4.1.2 Objective

This review synthesizes the empirical study by Liu et al. , which evaluates GPT-4 and Gemini-1.0 Pro on 180 real-world Java refactorings, situates it within the deep-learning-based and LLM-based refactoring literature, critically evaluates its methodology and findings, and identifies gaps for future research.

4.1.3 Relevance

Refactoring decisions remain a major bottleneck in software maintenance. Understanding whether LLMs can reliably identify and perform refactorings-and how their known unreliability (hallucination) can be mitigated-directly informs the safe integration of AI assistants into industrial development workflows.

4.2 Search Strategy (Methodology)

- **Databases:** Google Scholar, IEEE Xplore, ACM Digital Library, Scopus, and arXiv.
- **Keywords:** (“large language model” OR “LLM” OR “ChatGPT” OR “GPT-4” OR “Gemini”) AND (“software refactoring” OR “code smell” OR “refactoring opportunity” OR “extract method” OR “rename refactoring”); supplementary terms: “deep learning refactoring,” “refactoring detection,” “hallucination mitigation.”
- **Inclusion criteria:** peer-reviewed venues (ICSE, FSE, ASE, ISSTA, MSR, TSE, TOSEM) or widely cited preprints; publications from 2019–2024; studies on automated/semi-automated refactoring, LLM-based software engineering, or refactoring detection tooling; English language.
- **Exclusion criteria:** studies unrelated to source-code quality, purely opinion pieces, and work on non-behavior-preserving code transformation.

- **Snowballing:** backward citation tracing from the primary paper’s reference list identified foundational works (for example., Fowler’s refactoring catalog, RefactoringMiner, ReExtractor).

4.3 Thematic Synthesis

4.3.1 Theme 1: Deep Learning-Based Refactoring (Pre-LLM Era)

Early neural approaches targeted individual refactoring subtasks. Liu et al. pioneered deep-learning-based code smell detection, though their reliance on synthetic training data risked learning artificial rather than real smells-later mitigated by mining real-world refactorings. Other lines applied learned code representations (code2vec/code2seq) with classifiers to detect feature envy and recommend *move method* refactorings, exploited historical/structural metrics for god-class detection, and used NMT models to replicate developer code changes (up to 36% replicated). BERT- and T5-based models addressed rename suggestion (RefBERT) and readability improvement. **Consensus:** learned models can capture refactoring patterns, but each system handles only a narrow refactoring type and requires task-specific training data.

4.3.2 Theme 2: LLMs in General Software Engineering

LLMs have shown strong results in code generation, unit test generation (70.2% median statement coverage), program repair (162/337 Defects4J bugs fixed), fault localization, and vulnerability detection. Prompt engineering is repeatedly identified as decisive: prompt-pattern catalogs and concise scenario descriptions substantially improve outcomes, while non-determinism (mitigated by low temperature) threatens reproducibility. **Disagreement:** LLMs excel at some tasks yet underperform specialized models at others (for example., code summarization), so task-specific evaluation is required.

4.3.3 Theme 3: LLM-Based Refactoring

Exploratory studies mined developer–ChatGPT conversations and pull requests, confirming that practitioners already use LLMs for refactoring. Shirafuji et al. showed few-shot prompting reduces cyclomatic complexity while preserving semantics; PyCraft used LLM-generated variants to boost transformation-by-example techniques; EM-Assist combined LLM suggestions with IDE precondition checks and program slicing for *extract method* only. DePalma et al. found ChatGPT refactors effectively but offered no quantitative human-expert comparison. **Gap addressed by the primary study:** no prior work comprehensively and quantitatively compared LLM refactoring against human experts across multiple refactoring types, nor offered a general safety mechanism.

4.3.4 Methodological Approaches of the Primary Study

Liu et al. constructed *ref-Dataset*: 180 expert-validated real-world refactorings (9 types $\times$ 20 instances) mined from 20 popular Java projects using two detectors (ReExtractor, RefactoringMiner), restricted to post-April-2023 commits to avoid training-data contamination. A *reverse refactoring* technique produced inputs containing genuine refactoring opportunities plus their motivating changes. Three prompt templates (P1 generic; P2 type-specified; P3 solution-focused) were tested, with success measured per-entity or via a Dice-coefficient tolerance (≥ 0.5) for extract refactorings. Solution quality was rated by three experienced developers (Fleiss’ $\kappa = 0.82$) on a five-point scale (Excellent–Buggy), supported by a 102-unit-test suite.

4.3.5 Key Findings

- **Opportunity identification (RQ1):** with generic prompts, GPT-4 and Gemini identified only 28/180 (15.6%) and 7/180 (3.9%) opportunities. Specifying refactoring types raised GPT to 52.2% and Gemini to 21.1% (statistically significant, Wilcoxon $p < 0.001$). Adding refactoring *subcategories* (66.7%) and *search-space limitation* (86.7%) further improved GPT, reaching 100% for extract variable and inline method.
- **Code size effect:** success rate correlates negatively with LOC (point-biserial -0.38 for GPT under P2), motivating search-space narrowing.
- **Solution quality (RQ2):** 63.6% of GPT's 176 solutions (56.2% of Gemini's 137) were comparable to or better than developer solutions; 48 GPT solutions were identical to ground truth; renaming solutions were weakest (43.8% comparable-or-better).
- **Safety:** 13/176 GPT and 9/137 Gemini solutions were unsafe (18 semantic bugs, 4 syntax errors), motivating REFACTORINGMIRROR, a detect-and-reapply tactic that reapplies LLM-detected refactorings via IntelliJ IDEA. It reapplied 94.3% (33/35) of refactorings and eliminated *all* bugs.

4.4 Critical Evaluation

4.4.1 Gaps in Research

- No quantitative comparison of LLM-based refactoring against existing manual or semi-automated pipelines (smell detectors, EM-Assist, IntelliJ) in terms of quality gain versus cost.
- Cross-document refactorings (move class/method) remain unstudied due to LLM context-length limits.
- Only Java is covered, as state-of-the-art detection tools (ReExtractor, RefactoringMiner) do not support other languages; multi-language tools like RefDiff exhibit inferior precision.
- Neither the tactic nor detection tools can distinguish *profitless* safe refactorings (for example., renaming to "m") from beneficial ones-quality assessment of suggestions is an open problem.
- Practicality gap: developers rarely know the correct refactoring type/subcategory in advance, yet the highest success rates depend on this prior knowledge.

4.4.2 Contradictions and Debates

- *Definition of hallucination:* the primary study defines it as semantic/syntactic breakage, whereas EM-Assist defines it as violating IDE preconditions or trivial extractions-demonstrating an unsettled terminology in the field.
- *Model generality:* GPT dominates Gemini across all types, yet Gemini under a good prompt (P2) outperforms GPT under a poor one (P1), suggesting prompt quality can outweigh model choice; models may also be complementary on different refactoring types.

- *LLM capability profile*: strong LLM performance on repair and test generation contrasts with weakness on summarization and, here, on inline and extract-class identification-contradicting any blanket claim of LLM superiority.

4.4.3 Bias and Limitations

- **Sample size**: 180 instances (20 per type) is small; manual ground-truthing cost 42 man-days, limiting scale. Statistical power for per-type conclusions is thus modest.
- **Subjectivity**: expert validation and quality ratings are inherently subjective, though mitigated by independent triple rating, discarding disagreements, and high inter-rater agreement ($\kappa = 0.82$).
- **Stochasticity**: LLM outputs are non-deterministic, threatening reproducibility; the authors mitigate via sampling across projects but do not report repeated-run variance.
- **Data contamination controls** (post-cutoff commits) are sound but cannot fully rule out memorized project idioms.
- **Tool dependence**: REFACTORINGMIRROR inherits the blind spots of ReExtractor (missed 2/35 refactorings) and IntelliJ's supported refactoring catalog (1 unsupported type), bounding its coverage.
- The paper reports an internal inconsistency worth noting: the abstract cites an 86.7% best success rate while the conclusion states 86.1%.

4.5 Conclusion and Future Directions

4.5.1 Summary

The reviewed literature shows a clear progression from narrow, task-specific deep learning refactoring tools to general-purpose LLMs. Liu et al.'s study establishes that (1) LLMs are poor at unguided refactoring opportunity identification but reach 86.7% success with type-, subcategory-, and scope-constrained prompts; (2) roughly two-thirds of LLM refactoring solutions match or exceed human-expert quality; and (3) ~7% of solutions are unsafe, a risk fully neutralized by the detect-and-reapply tactic REFACTORINGMIRROR, which pairs LLM creativity with well-tested refactoring engines. The overarching implication is that current LLMs are best deployed as *suggestive assistants with mandatory post-processing*, not as autonomous refactoring engines.

4.5.2 Future Scope

Building on the identified gaps, future work can: (a) develop automated assessment of refactoring *benefit* to filter profitless suggestions; (b) extend evaluation to cross-document refactorings and languages beyond Java as context windows and detection tools mature; (c) empirically benchmark LLM pipelines against established semi-automated tools on cost-benefit terms; (d) explore retrieval-augmented few-shot prompting and interactive human-in-the-loop workflows; and (e) integrate LLM refactoring suggestions into code review, where human validation naturally guards against unsafe changes.

5 An Empirical Evaluation of Code Smell Detection in Angular Applications

5.1 Introduction

5.1.1 Topic Background

Modern web development is dominated by component-based JavaScript frameworks such as React, Angular, and Vue. While these frameworks improve performance, reusability, and modularity, they also introduce design flaws known as **code smells**—a term coined by Fowler to describe symptoms of poor code quality that may indicate deeper design problems. Code smells have been extensively studied in object-oriented systems (Java, C#) and, more recently, in front-end contexts such as plain JavaScript and React. However, **Angular**, despite being one of the most widely adopted frameworks for large-scale single-page applications (SPAs), has remained underexplored in terms of empirical evidence and detection tooling. Key Angular concepts relevant to this review include *components* (decorated TypeScript classes bound to HTML templates), *services and dependency injection*, *two-way data binding*, and *change detection strategies* (Default vs. OnPush).

5.1.2 Objective

This review synthesizes the contributions of Nunes et al. (2026), who (i) construct the first catalog of Angular-specific code smells derived from a grey literature review (GLR), (ii) compare Angular smells with existing React smell catalogs to identify cross-framework issues, and (iii) implement and empirically evaluate an automated static analysis tool that detects Angular code smells.

5.1.3 Relevance

Angular's rich feature set (modules, dependency injection, template syntax, RESTful handling) increases architectural complexity and the likelihood of design flaws. Systematic smell identification and automated detection can reduce technical debt, guide refactoring, support IDE-integrated real-time feedback, and serve pedagogical purposes in teaching software design. As the first Angular-specific smell catalog, this work fills a clear gap in front-end software quality research.

5.2 Search Strategy (Methodology of the Reviewed Study)

5.2.1 Databases and Sources

The authors conducted a **grey literature review** following Garousi et al.'s guidelines, since formal academic literature on Angular quality issues is scarce. The search was executed on **Google** (June 2025), restricted to English-language sources, and limited to the first 100 results as a stopping condition based on evidence exhaustion.

5.2.2 Keywords and Boolean Operators

The query combined the keyword "Angular" (AND) with OR-connected synonyms of code smell: "code smell(s)", "anti-pattern(s)", "antipattern(s)", "bad smell(s)",

"bad practice(s)", and hyphenated variants.

5.2.3 Inclusion and Exclusion Criteria

A quality assessment checklist evaluated (1) *source credibility* (publication/author linked to an organization working with Angular) and (2) *author expertise* (prior Angular publications, demonstrated experience); a source was accepted if it met at least one criterion. From **128 initial sources**, two authors excluded those failing quality criteria (26), unrelated to code smells or Angular (23), duplicates (10), covering deprecated AngularJS (5), or discussing only traditional smells (5), yielding **58 sources (S1–S58)**. Thematic analysis produced 52 candidate smells; candidates were retained only if they matched a previously cataloged React smell (Ferreira & Valente) *and/or* appeared in at least two distinct sources, resulting in **11 final smells**. Context-sensitive candidates (for example., *Default Change-Detection Strategy*, *No trackBy in ngFor*, *Nested Subscriptions*) were excluded as library-specific or project-dependent rather than universal design flaws.

5.3 Thematic Synthesis

5.3.1 Theme 1: The Angular Code Smell Catalog (RQ1)

Eleven smells were identified and classified as *Angular-specific* or *cross-framework*: **Large Component** (6 sources), **Inefficient Method Binding in Templates** (5), **Overusing Any Type** (4), **Excessive Parent-to-Child Communication** (3), **Direct DOM Manipulation** (3), **Coupled Services** (2), **Inheritance Instead of Composition** (2), **Too Many Inputs** (2), **Prop Drilling** (1), **Large File** (1), and **Duplicated Component** (1). Recurring underlying causes include violation of the Single Responsibility Principle (Large Component, Coupled Services—the latter resembling a *God Class* at the service layer), tight coupling (@ViewChild misuse, inheritance hierarchies, prop drilling via chained @Input/@Output), erosion of type safety (any overuse), and bypassing Angular’s declarative paradigm (direct DOM manipulation via ElementRef; method calls in templates that re-execute on every change-detection cycle).

5.3.2 Theme 2: Community Knowledge Sources (RQ2)

Smells are predominantly discussed in informal, community-driven venues: personal blogs (21), forum posts (15), corporate blogs (10), course material (5), videos (3), Q&A (2), and official documentation (2). Notably, a single Reddit thread with 53 comments yielded 10 of the 11 smells, underscoring the value of practitioner discussion as experiential knowledge that formal documentation overlooks.

5.3.3 Theme 3: Cross-Framework Generalization (RQ3)

Six smells overlap with React’s catalog—Large Component, Direct DOM Manipulation, Inheritance Instead of Composition, Too Many Inputs (React: *Too Many Props*), Prop Drilling, Large File, and Duplicated Component—indicating that many design problems are properties of component-based architectures generally rather than of any single framework. Conversely, React smells tied to its rendering model (*Force Update*, *Props in Initial State*, *Uncontrolled Components*, *JSX Outside Render Method*, *Low Cohesion*) did not appear in Angular literature, confirming that framework abstractions shape smell manifestation.

5.3.4 Theme 4: Methodological Approaches to Detection (RQ4)

The proposed tool integrates two existing detectors-**ReactSniffer** (React-specific) and **SniffTSX** (TypeScript-specific, applicable to both React and Angular)-in a pipeline of *File Reader* → *AST Extractor* (Babel parser) → *Controller* → *Analyzers* → *CLI Output*. The framework is auto-detected from file extensions and `package.json`, and detectors are selectively activated. Because syntactic realizations differ (for example., `@Component`-decorated classes vs. React functional components), cross-framework smells share detection logic but require framework-specific AST implementations; each detector is a pure function with a consistent signature, keeping the architecture extensible. The prototype implements five analyzers: Overusing Any Type, Large Component, Large File, Inheritance Instead of Composition, and Too Many Inputs.

5.3.5 Theme 5: Key Findings on Detection Effectiveness (RQ5)

Evaluation used ten popular open-source Angular GitHub projects. Since no labeled dataset existed, the authors built one: 30 manually validated instances per smell (150 total), each paired with a manually refactored, smell-free variant to test for false positives. Results (Accuracy / Precision / Recall / F1): *Overusing Any Type* 1.00/1.00/1.00/1.00; *Inheritance Instead of Composition* 1.00/1.00/1.00/1.00; *Too Many Inputs* 1.00/1.00/1.00/1.00; *Large File* 0.95/0.90/1.00/0.95; *Large Component* 0.88/0.81/1.00/0.89. All smells achieved perfect recall; precision losses were confined to size-based smells, whose absolute lines-of-code thresholds flag cohesive but legitimately large components as false positives.

5.4 Critical Evaluation

5.4.1 Gaps in Research

Only 5 of the 11 cataloged smells have implemented detectors; dynamic/behavioral smells (for example., Inefficient Method Binding, Coupled Services, Prop Drilling) remain undetected. The prevalence and *practical impact* of the smells in real-world codebases were not measured-the evaluation dataset was curated rather than exhaustively mined. Developer perception (via surveys or interviews, as done in the React studies by Ferreira & Valente and Nunes et al.) was not triangulated. Other component frameworks (Vue, Svelte) are hypothesized but not studied.

5.4.2 Contradictions and Debates

Several candidate smells (Default Change-Detection Strategy instead of OnPush, No trackBy in ngFor, Unoptimized Module Loading) provoked disagreement among the authors themselves: they affect performance yet are context-dependent (for example., Micro Frontend constraints), illustrating an unresolved debate about whether performance-related practices constitute universal design smells. The size-threshold results also expose a tension between simple metric-based detection and conceptual definitions grounded in cohesion and responsibility.

5.4.3 Bias and Limitations (as acknowledged by the authors)

- **Construct validity:** grey literature is not peer-reviewed and uses inconsistent terminology; mitigated by aligning findings with academic smell definitions.
- **Conclusion validity:** qualitative manual classification risks subjectivity; mitigated by iterative multi-author review and source traceability.

- **Internal validity:** relevant sources may have been missed despite a systematic, documented search limited to Google's first 100 results (a single search engine is itself a coverage risk).
- **External validity:** findings are Angular- and grey-literature-specific and may not generalize to industrial practice or other frameworks; smell relevance varies with application scale and context.
- Additional reviewer concerns: single-engine search, English-only sources, low source counts for some smells (three smells rest on only 1–2 sources), and a balanced curated dataset that likely overstates real-world precision.

5.5 Conclusion and Future Directions

5.5.1 Summary

Nunes et al. deliver the first evidence-based catalog of 11 Angular code smells, show that six transcend framework boundaries, and demonstrate that a unified, extensible static-analysis tool can detect five of them with high effectiveness (accuracy ≥ 0.88 ; F1 0.89–1.00). The work establishes that community grey literature is a rich, credible source of front-end quality knowledge and that framework-specific syntactic adaptation is necessary even for conceptually shared smells.

5.5.2 Future Scope

The authors plan to (i) implement detectors for the remaining smells, (ii) triangulate findings with empirical analyses of real-world Angular codebases to measure prevalence and impact, (iii) evaluate the tool in mixed-framework projects and improve reporting, and (iv) extend the methodology to Vue, Svelte, and other frameworks toward a unified front-end smell catalog. Building on these gaps, subsequent research could refine size-based detectors with cohesion-aware metrics to reduce false positives, incorporate developer surveys to validate perceived smell severity, and explore machine-learning or LLM-based detection as a complement to rule-based AST analysis.

6 Patterns of Developer Adoption of LLM-Generated Code Refactoring Suggestions

6.1 Introduction

6.1.1 Topic Background

Code refactoring improves the internal structure of code without changing its external behavior. LLMs such as ChatGPT are increasingly used to generate refactoring suggestions, but prior research has focused on evaluating the quality of these suggestions in isolation rather than how developers actually integrate them into real codebases.

6.1.2 Objective

The paper investigates how developers adopt ChatGPT-generated refactoring suggestions in practice, by (i) quantifying the degree of adoption using similarity metrics and (ii) qualitatively identifying the patterns of modification developers make before committing.

6.1.3 Relevance

Because refactoring must preserve behavior, understanding how much correction developers actually apply to LLM output is essential for judging its real usefulness, deciding which refactoring tasks can be automated, and informing AI-assisted development workflows.

6.2 Dataset and Methodology

6.2.1 DevGPT Dataset

The study builds on DevGPT, a dataset linking 29,778 ChatGPT prompts/responses to GitHub artifacts (commits, issues, PRs) collected during July–August 2023, covering GPT-3.5 and GPT-4.

6.2.2 Keyword-Based Filtering

From 3,245 candidate commits, the authors compile 25 refactoring-related keywords (for example, *refactor*, *rename*, *restructure*, *simplify*, *clean*, *split*) drawn from refactoring taxonomies in the literature and terms empirically extracted from commit messages, narrowing the dataset to 169 commits.

6.2.3 Data Extraction and Mapping

Using the GitHub API, the authors reconstruct before/after file states from unified diffs, extract ChatGPT-suggested code blocks from linked conversations, and map each changed file to its most similar suggestion via normalized Levenshtein similarity, producing 440 datapoints.

6.2.4 Similarity Metrics

Adoption is measured with Jaccard n -gram similarity, normalized Levenshtein similarity, a newly introduced Token Match Rate (to handle suggestions spanning multiple files), and CrystalBLEU for semantic-level similarity.

6.3 Findings: Adoption of Refactoring Suggestions (RQ1)

6.3.1 Prompt Position of Adopted Suggestions

Developers typically reach their refactoring goal within the first 1–4 prompts of a conversation; longer conversations are usually tied to multi-file refactoring requests rather than repeated failure.

6.3.2 Bimodal Similarity Distribution

Jaccard, Levenshtein, and CrystalBLEU scores show a bimodal pattern: developers either apply suggestions with almost no changes (high similarity) or make major modifications (low similarity), with few cases of minor edits.

6.3.3 Token Match Rate Insight

Token Match Rate scores are concentrated at high values, showing that even when parts of a suggestion are deleted, the retained content is still largely drawn from the LLM's suggestion rather than rewritten independently.

6.4 Findings: Types of Modifications (RQ2)

6.4.1 Refactoring Goals Observed

Manual inspection of 190 datapoints shows developers most often seek improved readability (renaming, documentation, cleanup) and maintainability (restructuring, splitting logic, adopting OOP), with simplification as a secondary goal.

6.4.2 Five Adoption Patterns

The authors identify five recurring patterns: (1) full adoption without modification, often via automated prompt-to-commit pipelines; (2) omission of erroneous or unwanted new behavior while keeping the rest; (3) minor integration fixes due to ChatGPT's limited project context; (4) restructuring the suggested logic to fit existing design conventions; and (5) reusing only the suggestion's structure or only its content, disregarding the other.

6.4.3 Automation-Ready vs. Context-Dependent Tasks

Low-risk, well-scoped tasks (renaming, file reorganization, documentation) are adopted almost verbatim, often through automated scripts, while complex tasks (new software designs, cross-file logic splitting) require substantial developer correction.

6.5 Critical Evaluation and Conclusion

6.5.1 Contributions and Implications

The paper's main contributions are an empirical account of real-world LLM refactoring adoption and the Token Match Rate metric; implications include that low-risk refactoring is increasingly automatable, while validation and testing infrastructure remain necessary for complex refactoring automation.

6.5.2 Limitations

The dataset only captures *adopted* suggestions, since inclusion requires a linked ChatGPT conversation in the commit message, so rejected suggestions are invisible; it is also imbalanced, with one repository (*tisztamo/Junior*) contributing 143 of 169 commits, and is limited to the GPT-3.5/GPT-4 era.

6.5.3 Future Directions

The authors call for extending adoption studies to other LLMs and larger, more balanced datasets, investigating the role of automated testing in enabling safer refactoring automation, and exploring prompt-traceability tooling as AI-assisted contributions become more common in software repositories.

7 A Systematic Literature Review of Agentic AI: Definitions, Architectures, and Challenges

7.1 Introduction

7.1.1 Topic Background

Agentic AI refers to systems that do not merely react to inputs but actively pursue objectives, adapt strategies, and initiate actions in dynamic environments, distinguishing it from earlier AI paradigms built primarily for narrow, human-supervised tasks. The paper traces the concept of agency back to philosophy, psychology, and the social sciences, and shows that its translation into AI remains contested: some researchers equate Agentic AI with autonomous agents or multi-agent systems, while others treat it as an emergent property of large language models (LLMs) embedded in feedback loops, memory structures, or tool-use frameworks.

7.1.2 Objective

The paper conducts a Systematic Literature Review (SLR) following Kitchenham's methodology, guided by one main question (*what are the general approaches to Agentic AI research?*) and four sub-questions covering definitions, architectures, application domains, and open risks/-gaps. Its central methodological contribution is an *Intersection Analysis* that maps each study's contribution type against the research gap it addresses, used to identify areas of concentration versus neglect in the field.

7.1.3 Relevance

Given the rapid, fragmented growth of Agentic AI research across cognitive architectures, reinforcement learning, LLM-based agents, and human-AI collaboration, a structured synthesis is needed to clarify definitional ambiguity, consolidate scattered architectural knowledge, and surface concrete engineering and safety challenges before Agentic AI systems are deployed more broadly in real-world, high-stakes settings.

7.2 Search Strategy (Methodology)

7.2.1 Databases and Sources

The search was conducted exclusively in databases widely used in engineering and computing research: ACM Digital Library, Elsevier, IEEE Xplore Library, Nature, ScienceDirect, and Springer Link, in line with Kitchenham's SLR methodology and the PRISMA 2020 reporting principles.

7.2.2 Keywords

The search string *Title: "agentic ai" OR "AI agent" OR "autonomous agent" AND ("LLM" OR "large language model")* was applied to titles and full texts, with syntax adapted per database, and iteratively refined based on preliminary outcomes.

7.2.3 Inclusion and Exclusion Criteria

The initial search (2021–mid-2025) returned 125 studies. After removing studies not published in journals/proceedings and those unrelated to Agentic AI (EC1, EC2), 115 remained; after deduplication across databases (EC3), 109 remained. Manual screening by title/abstract and, where inconclusive, full text, performed independently by two researchers with a third resolving disagreements, produced a final corpus of 48 studies, each further scored against seven qualitative evaluation (QE) questions covering purpose, context, related work, methodology, results, conclusions, and future work.

7.3 Thematic Synthesis

7.3.1 Theoretical Foundations of Agency and Agentic AI

The paper distinguishes autonomy (operational independence without goals or planning) from agency (goal-directed, intentional behavior), and traces the concept through philosophy (Dennett's intentional stance, Gunkel's machine ethics), sociology (Giddens' structuration theory), and psychology (Bandura's theory of human agency), before showing how these ideas map onto competing AI definitions: Agentic AI as synonymous with autonomous agents versus Agentic AI as an emergent property of tool-augmented LLMs.

7.3.2 Architectural Evolution: From Symbolic Systems to LLM-Based Agents

The review traces architectural progress from symbolic/GOFAI expert systems (e.g., MYCIN, DENDRAL), through Belief-Desire-Intention (BDI) architectures, to modern Multi-Agent Systems (MAS) and reinforcement learning agents, culminating in LLM-based agents (e.g., AutoGPT, BabyAGI) capable of task decomposition, tool use, and long-term planning despite lacking explicit symbolic architectures.

7.3.3 Proposed Taxonomy of Research Fields

Based on the 48 selected studies, the authors propose a five-field taxonomy: Memory & Cognition (4.3%), Networking & Systems (27.7%), Trust & Safety (8.5%), Evaluation & Limits (27.7%), and Applications & Use-Cases (31.9%), the last being the most heavily researched area.

7.3.4 Domain Applications Across the Corpus

Applications identified span healthcare (Alzheimer's risk prediction, cancer diagnosis, clinical calculations), industry and manufacturing (Industry 6.0, supply chains, robot control), agriculture (crop classification, disease detection), finance, cybersecurity, and education, illustrating a research landscape currently prioritizing deployment of existing agentic capabilities over foundational innovation.

7.3.5 Bibliometric Trends

Publication volume rose sharply from 2021 to 2025, concentrated heavily in the United States (21 papers), followed by India, China, and the UK, indicating a globally uneven research landscape with a small number of dominant hubs.

7.4 Critical Evaluation

7.4.1 The Intersection Analysis: Contribution vs. Gap Mapping

The paper’s heatmap analysis, mapping contribution types (survey, framework, method, model, evaluation, application, theory) against research gaps (adaptability, scalability, autonomy, generalization, trust, efficiency, human-AI collaboration), reveals research is concentrated at the intersection of *Application* contributions and the *Autonomy* gap, indicating the field is in an “exploitation” phase focused on applying existing autonomous capabilities to new domains.

7.4.2 The Adaptability and New-Models Research Desert

The central critical finding is a “research desert” around Adaptability: only two studies primarily address it, neither offering a constructive framework, model, method, or application. Similarly, the New Models gap is almost entirely unaddressed, with existing work limited to surveys or evaluations of established paradigms rather than proposals for fundamentally new architectures.

7.4.3 Limitations of the Review Itself

The SLR’s exclusion criteria (EC1–EC3) rely partly on manual screening, introducing potential reviewer bias despite dual independent screening; the corpus is also geographically skewed toward a handful of countries, and the taxonomy’s five-field structure, while integrative, necessarily compresses heterogeneous subfields (e.g., MAS and LLM agents) into shared categories that may obscure finer distinctions.

7.4.4 Comparison with Prior Surveys

Unlike prior narrative surveys on autonomous agents or domain-specific reviews (e.g., Industry 4.0), this is presented as the only PRISMA-compliant SLR combined with a quantitative Intersection Analysis capable of identifying the Adaptability Desert, positioning its methodological rigor as a distinguishing contribution over existing literature.

7.5 Conclusion and Future Directions

7.5.1 Summary

The field has rapidly shifted from conceptual, theory-driven work toward operational deployment, with research heavily prioritizing Autonomy and Applications while Adaptability and New Modeling paradigms remain critically underexplored, leaving current agentic systems vulnerable to brittleness outside controlled environments.

7.5.2 Future Scope

The authors propose a concrete research agenda: developing “Metacognitive Architectures” with self-reflection and real-time plan modification, shifting evaluation from static accuracy metrics toward “Resilience Benchmarks” (e.g., recovery rate from induced errors via Chaos Engineering), and prioritizing “Adaptive Learning” frameworks that update agent knowledge post-deployment without full retraining.

8 Exploring the Use of Large Language Models for Data Extraction for Systematic Reviews in Software Engineering

8.1 Introduction

8.1.1 Topic Background

Systematic reviews in Software Engineering (SE), including systematic literature reviews, systematic mapping studies (SMS), and tertiary studies are widely used to aggregate evidence on a research topic, but require a rigorous, multi-step process. Among these steps, data extraction is particularly effort-intensive and has historically received the least automated support, despite growing interest in using Large Language Models (LLMs) to automate it in other fields.

8.1.2 Objective

Building directly on Felizardo et al., the first SE-specific study of LLM-based data extraction, this paper evaluates GPT-4o's performance in extracting data from 46 primary studies of an existing SMS on software issue report classification, comparing automated extraction against manually extracted ground truth.

8.1.3 Relevance

Establishing how reliably an LLM can perform data extraction matters because it determines whether such tools can be trusted to reduce the manual burden of systematic reviews in SE, and, if so, what workflow (fully automated versus human-in-the-loop) is appropriate given current accuracy levels.

8.2 Background: Task and Replicated Study

8.2.1 The Data Extraction Task

An SMS aims to categorize and quantify existing literature on a research area to identify trends, methodologies, and gaps. Its data extraction step involves systematically pulling structured information from primary studies, and remains largely manual and under-tooled compared to other steps such as search and study selection.

8.2.2 The Replicated SMS

The study replicates the data extraction step of an existing manually conducted SMS on issue report classification, a SE task that categorizes reported issues as bugs (requiring code changes) or non-bugs (for example., feature requests, questions), which supports more efficient defect management and resource prioritization.

8.3 Methodology

8.3.1 Extraction Setup

GPT-4o was provided the 46 primary study PDFs in batches and prompted with a structured data extraction template covering five research questions: (RQ1) proposed classification techniques,

(RQ2) features used, (RQ3) pre-processing/tokenization techniques, (RQ4) study context (open- vs. closed-source data), and (RQ5) whether practitioners were involved for feedback.

8.3.2 Scoring Procedure

Each GPT-4o response was scored against manually extracted ground truth on a graded scale: 1 if all identified items were correct, 0.75 if more than half were correct, 0.5 if exactly half, 0.25 if less than half, and 0 if none were correct. Each RQ's final score was computed as the sum of per-study scores divided by the maximum possible score (46).

8.4 Results

8.4.1 Overall Performance

GPT-4o achieved an overall average accuracy of 79% across the five research questions, indicating meaningful but incomplete reliability for automated extraction.

8.4.2 Per-Question Variation

Performance varied substantially by question type: RQ4 (study context) scored highest at 98%, near-perfect; RQ5 (practitioner involvement) and RQ2 (features used) scored 84% and 77% respectively; RQ1 (classification techniques) scored 75%; and RQ3 (pre-processing/tokenization techniques) scored lowest at 64%, indicating this type of fine-grained technical detail was most challenging for the model to extract accurately.

8.5 Discussion, Threats, and Conclusion

8.5.1 Interpretation and Recommended Workflow

Given the 79% average accuracy and the substantial variability across question types, the authors argue GPT-4o cannot fully automate data extraction but can meaningfully support a semi-automated workflow: initial LLM-based extraction, followed by manual validation on a sample to identify common errors, iterative prompt refinement, and then scaled application with human review.

8.5.2 Validity Threats

Key threats include reliance on a single SMS with only 46 primary studies and evaluation of a single LLM (GPT-4o), limiting generalizability; use of human-extracted data as ground truth, which could carry annotator bias (though mitigated by involving two researchers); and the stochastic nature of LLM outputs, which may affect reproducibility of the reported scores.

8.5.3 Conclusion and Future Work

The study concludes that GPT-4o is a useful but imperfect aid for SMS data extraction, best used within a human-in-the-loop pipeline rather than as a full replacement. Future work will extend evaluation to additional LLMs (Gemini, Llama, DeepSeek), apply the approach to other SE domains (effort estimation, code quality, defect prediction), adopt more objective automated evaluation metrics (for example., n-gram or LLM-as-judge comparisons) in place of manual scoring, and systematically study the effect of prompt engineering on extraction accuracy.

9 CodeSense: A Static Analysis Tool for Detecting Code Smells

9.1 Introduction

9.1.1 Topic Background

Code smells are characteristics in source code that, while not bugs or functional errors, signal underlying design or implementation problems that negatively affect maintainability and increase technical debt. Conventional static analysis tools rely on predefined rules and metric thresholds, which detect many simple violations but fail to capture complex, context-dependent smells due to their lack of a global, semantic view of the codebase.

9.1.2 Motivation

Machine learning (ML) approaches to code smell detection can capture more complex patterns than rule-based tools but are computationally expensive, since they typically require training a separate model for each type of smell, limiting their scalability. Recent advances in LLMs and Generative AI suggest new possibilities for contextual code understanding, motivating the design of a tool that pursues similar contextual awareness without relying on large pre-trained language models at inference time.

9.1.3 Contributions

The paper proposes CodeSense, a hybrid static analysis tool combining rule-based, pattern-based, and heuristic-based detection to achieve context-aware code smell detection. The authors claim CodeSense improves on existing tools' accuracy while avoiding the computational cost of full ML-model-per-smell approaches, reporting an average F1-score of 0.87 against baseline tools SonarQube (0.75), PMD (0.71), and Checkstyle (0.66).

9.2 Background and Related Work

9.2.1 Taxonomy of Code Smells

Drawing on Fowler's classification, the paper groups common code smells into four categories: Bloaters (for example., Long Method, Large Class, Long Parameter List), Object-Orientation Abusers (for example., Switch Statements, Refused Bequest), Change Preventers (for example., Divergent Change, Shotgun Surgery), and Dispensables (for example., Duplicate Code, Dead Code, Data Class), noting that Long Parameter List and Complex Method are especially common and impactful in real projects.

9.2.2 Evolution of Detection Techniques

Code smell detection has progressed from manual, time-consuming reviews to automated rule- and metric-based techniques (for example., using Cyclomatic Complexity, LCOM, DIT) implemented in tools such as SonarQube, PMD, and Checkstyle, and more recently to ML-based approaches (CNNs, RNNs, autoencoders) that reduce dependence on fixed thresholds but introduce high computational and data-labeling costs; LLM-based approaches are an emerging but still supplementary direction.

9.2.3 Research Gaps

The paper identifies four gaps motivating CodeSense's design: limited contextual understanding in rule/metric-based tools (leading to false positives), the high computational cost of training ML models per smell type, limited adaptability of rule-based systems to new or evolving smells, and a general lack of actionable insight existing tools detect smells but rarely explain why they exist.

9.3 System Design: The CodeSense Architecture

9.3.1 Four-Module Pipeline

CodeSense is structured into four modules: the Code Ingestion Module (CIM), which parses source code into Abstract Syntax Trees (ASTs); the Program Analysis Module (PAM), which performs Control Flow Analysis, Data Flow Analysis, Dependency Analysis, and constructs a unified Semantic Graph; the Code Smell Detection Module (CDM), which applies rule-based, pattern-based, and heuristic-based detection; and the Reporting and Visualization Module (RVM), which generates context-aware reports with refactoring recommendations.

9.3.2 Hybrid Detection Strategy

The CDM combines three detection strategies: rule-based detection for structural smells (for example., Long Method, Large Class) using configurable thresholds; pattern-based detection for complex/contextual smells (for example., Feature Envy) via pattern matching on the semantic graph; and heuristic-based detection, using expert-derived heuristics and lightweight ML classifiers, for smells lacking explicit rules or patterns.

9.3.3 Key Design Innovations

The authors highlight four design principles: contextual awareness via semantic and dependency analysis to reduce false positives; the hybrid detection strategy itself, to cover both simple structural and complex design smells; an extensible modular architecture allowing new languages or detection methods to be added; and actionable insights, where the tool explains why a smell exists rather than only flagging it.

9.4 Implementation

9.4.1 Technology Stack

CodeSense is implemented primarily in Python. The CIM uses tree-sitter for fast, incremental AST/CST generation across languages and pygments for tokenization; the PAM uses custom AST traversal plus networkx to build and query CFGs, DFGs, and the semantic graph; the CDM combines a custom declarative rule engine, networkx-based graph pattern matching (for example., subgraph isomorphism), and scikit-learn classifiers (SVM, Random Forests) for heuristic smells; and the RVM uses Seaborn/Matplotlib plus custom HTML/CSS templates for interactive reporting.

9.4.2 Technical Challenges

Key challenges addressed include parsing performance and robustness to incomplete/erroneous code (solved via tree-sitter and AST caching), scalability for large codebases (via incremental

analysis of changed code segments), bridging the semantic gap between syntax and meaning (via the semantic graph and custom data/control flow analyzers), balancing false positives against false negatives through iterative refinement, and presenting actionable, context-rich reports rather than bare smell lists.

9.5 Performance Evaluation

9.5.1 Dataset and Ground Truth

CodeSense was evaluated on a mix of open-source Java projects (for example., Apache Commons Lang, Google Guava, JFreeChart) and purpose-built synthetic codebases containing known instances of specific smells. Ground truth labels were established through manual inspection by software engineers cross-checked against existing static analysis tools.

9.5.2 Evaluation Metrics and Comparison Baselines

Performance was measured using standard Precision, Recall, F1-score, and Accuracy metrics, comparing CodeSense against three baseline tools - SonarQube, PMD, and Checkstyle - across five code smells (Long Method, Large Class, Data Class, Multifaceted Abstraction, Feature Envy), following a controlled procedure of tool configuration, codebase analysis, result mapping to ground truth, and metric aggregation.

9.5.3 Results

CodeSense achieved higher F1-scores than all three baselines on every evaluated smell, with an average F1-score of 0.87 versus 0.75 (SonarQube), 0.71 (PMD), and 0.66 (Checkstyle). Gains were modest for well-defined structural smells (Long Method, Large Class) but substantially larger for context-dependent smells such as Data Class, Multifaceted Abstraction, and Feature Envy, which the authors attribute to CodeSense's semantic graph and hybrid detection core.

9.6 Discussion and Conclusion

9.6.1 Interpretation of Results

The larger performance gap on smells requiring deeper understanding of data flow, class relations, and behavioral patterns supports the paper's hypothesis that combining program analysis with hybrid detection outperforms purely rule- or metric-based tools, with claimed downstream benefits including improved maintainability, reduced technical debt, higher developer productivity, and better design decisions.

9.6.2 Limitations

The authors acknowledge several open limitations: incomplete contextual coverage across all programming paradigms and domain-specific logic; potential performance bottlenecks on extremely large codebases despite incremental analysis; the inherent false positive/negative trade-off requiring ongoing rule and model refinement; current restriction to Java, limiting generalizability; lack of IDE/CI-CD integration; absence of dynamic analysis to catch runtime-only smells; and only conceptual (not deeply implemented) use of more sophisticated ML methods such as deep learning or GNNs.

9.6.3 Future Work

Planned extensions include enhanced semantic analysis methods (potentially with formal methods or theorem proving), integration of dynamic analysis alongside static analysis, automated (not just suggested) refactoring using LLM code-generation capabilities, broader language support beyond Java, performance optimization for large-scale systems via distributed/parallel computation, and larger-scale experimental validation on real-world datasets against established benchmarks.

10 Code Smell Detection Research Based on Pre-training and Stacking Models

10.1 Introduction

10.1.1 Topic Background

Code smell detection has traditionally relied on heuristic-based rules, machine learning, or deep learning approaches. Early tools such as JDeodorant and iPlasma used heuristic rule methods with manually set thresholds, but different tools' inconsistent thresholds led to high false positive rates, motivating the shift toward machine learning and, more recently, deep learning approaches.

10.1.2 Motivation

The paper identifies three limitations in existing code smell detection work: machine learning-based methods tend to use a single model rather than combining multiple models' strengths; existing datasets mostly capture only structural features, ignoring textual features in the code; and structural feature selection often overlooks relationships between different code smells, relying only on a subset of effective features.

10.1.3 Contributions

The paper proposes SCSmell, a code smell detection method combining structural features (via static analysis) with textual features (via a BERT pre-training model), feeding the combined representation into a three-layer Stacking model. Its three stated contributions are: (1) a new dataset built from 44 large-scale real-world projects with both structural and textual features, (2) the SCSmell detection method itself, and (3) a comparative validation against six machine learning models and six deep learning-based methods.

10.2 Method: The SCSmell Framework

10.2.1 Overview of the Pipeline

SCSmell first extracts structural features from 44 projects using static analysis tools, then converts the code to text format and extracts textual features via BERT, reducing the resulting 768-dimensional vector to one dimension using LDA. The structural and textual features are combined into labeled data samples, filtered using RFECV to select the top 10 features, balanced using Borderline-SMOTE, and finally classified using a three-layer Stacking model.

10.2.2 Data Collection

The dataset consists of 44 real-world GitHub projects (for example., Cayenne, Batik, Hadoop, JUnit, Xerces), with twelve exceeding 10K lines of code, spanning a combined total of roughly 28,798 classes, 252,340 methods, and 2.8 million lines of code.

10.2.3 Textual Feature Extraction

Source code is converted to text format and encoded using a pre-trained BERT model via the SentenceTransformer library; word vectors from the final hidden layer are averaged via pooling

into a fixed-length vector, which is then reduced from 768 dimensions to a single dimension using LDA.

10.2.4 Structural Feature Extraction

Structural features are extracted using iPlasma, with 24 metrics used to evaluate BrainClass, DataClass, and GodClass smells, and 21 metrics used for BrainMethod, covering metrics such as Coupling Between Objects (CBO), Cyclomatic Complexity (CC), and Access to Foreign Data (ATFD).

10.2.5 Feature Selection and Sample Balancing

The RFECV (Recursive Feature Elimination with Cross-Validation) method is used to select the top 10 most important structural features per code smell type, reducing overfitting and improving generalization. Borderline-SMOTE is then applied to address class imbalance, raising the proportion of positive samples in the dataset from 22% to 42%.

10.2.6 The Three-Layer Stacking Model

The classification model extends a traditional two-layer Stacking approach into three layers (primary, intermediate, advanced). The primary layer uses SVM, Logistic Regression, and Random Forest (the SLR combination, selected after comparing six base-classifier combinations); the intermediate layer combines Random Forest output with a model aggregator; and the advanced layer produces the final prediction. Experiments confirm the three-layer configuration outperforms a two-layer version across all metrics.

10.3 Experimental Setup

10.3.1 Research Questions

The evaluation is structured around five research questions: (RQ1) overall detection effectiveness of SCSmell, (RQ2) whether combining structural and textual features outperforms either alone, (RQ3) comparison against traditional ML models, (RQ4) comparison against deep learning models, and (RQ5) time performance of the detection pipeline.

10.3.2 Evaluation Metrics and Target Smells

Performance is measured using Accuracy, Precision, Recall, and F1-score across four code smell types: Brain Class (BM), Data Class (DC), God Class (GC), and Brain Method (BC).

10.4 Results

10.4.1 RQ1: Overall Detection Effectiveness

Evaluated on seven representative large programs (Batik, Xalan, HSQLDB, Argouml, Mylyn, Parallelcolt, Xerces), SCSmell achieved consistently high accuracy, precision, recall, and F1-scores across all four smell types, with performance dips in specific programs (for example, Xalan for BM) attributable to smaller sample sizes of that smell type in that project.

10.4.2 RQ2: Contribution of Combined Features

Datasets combining structural and textual features substantially outperformed datasets using either feature type alone, with recall improvements of roughly 17–21 percentage points over textual-features-only datasets, confirming that structural and textual features are complementary rather than redundant for detection accuracy.

10.4.3 RQ3: Comparison with Traditional ML Models

SCSmell outperformed six traditional ML baselines (Random Forest, Decision Tree, KNN, SVM, Logistic Regression, Adaboost) across all four smell types, with especially large margins over SVM (for example., roughly 33–49 percentage points higher on DC metrics); Adaboost was the closest competitor but was noted as computationally expensive.

10.4.4 RQ4: Comparison with Deep Learning Models

SCSmell also outperformed six deep learning baselines (BP, GRU, ResNet, DeepFM, DeepSmell, DeleSmell), with the largest gains over BP and GRU (up to roughly 19 percentage points) and smaller but consistent gains over the strongest deep learning baseline, DeleSmell. The authors attribute this to the Stacking model's reduced overfitting and its ability to integrate multiple base learners' predictions, versus deep learning's greater susceptibility to overfitting/underfitting on this data.

10.4.5 RQ5: Time Performance

Detecting brain method smells across the dataset took a total of 121 minutes (about 2 minutes 45 seconds per project on average), with static-analysis-based metric generation (39 min) and the SCSmell testing step itself (49 min, over 40% of total time) being the dominant cost centers, while text information generation was comparatively fast (8 min total).

10.5 Discussion and Conclusion

10.5.1 Threats to Validity

The authors identify two main threats: high code complexity in analyzed projects can increase the time cost of detection (an internal validity threat, to be addressed via future modular decomposition of the code), and the large dataset size increases computational and storage demands, potentially affecting performance and efficiency (an external validity threat, to be addressed via future distributed computing).

10.5.2 Conclusion

SCSmell combines structural and textual features via BERT-based text encoding and a three-layer Stacking model, achieving a 10.38% average precision improvement over existing detection methods while maintaining high precision, recall, and F1-scores across brain class, data class, god class, and brain method smells on 44 real-world projects.

10.5.3 Future Work

The authors plan to extend SCSmell's evaluation to additional code smell types beyond the four studied, and to develop visual charts or illustrative examples to make test results easier to

interpret.

11 EnseSmells: Deep ensemble and programming language models for automated code smells detection

11.1 Introduction

A code smell is a surface-level indicator of a deeper design or implementation problem in software source code - not a bug per se, but a symptom that raises the likelihood of future faults, poor maintainability, and higher technical debt (Fowler, 1999). Code smells are typically categorized by granularity into *implementation* smells (for example., Long Method), *design* smells (for example., God Class, Data Class), and *architectural* smells (for example., God Component) (Suryanarayana et al., 2014; Garcia et al., 2009). Detecting them early in the development lifecycle is widely regarded as good practice for controlling technical debt.

Objective. This review synthesizes the body of research that the EnseSmells paper (Ho, Bui, Nguyen, Di Salle & Le, 2025) builds on and positions itself against - spanning metric/rule-based detection, classical machine learning, and deep learning/embedding-based approaches - and examines how EnseSmells' proposed two-tier architecture (statistical semantic features + structural/metric features) advances this line of work.

Relevance. Code smell detection sits at the intersection of software engineering and machine learning. As codebases grow and review cycles shorten, manual or purely rule-based detection becomes impractical, while machine-learning approaches raise open questions about which representations (raw metrics vs. learned code embeddings) best capture the symptoms of specific smell types. Understanding this trajectory clarifies why a hybrid model like EnseSmells (combining CNN/LSTM-based semantic modeling with metric-based structural modeling) is a natural next step, and it highlights the practical stakes: better detection tools translate directly into more maintainable, less fault-prone software.

11.2 Search Strategy (Methodology)

This review is derived from the reference list and related-work discussion contained within the EnseSmells paper itself, rather than an independent multi-database search. As reported by the authors:

- **Sources used by the paper:** peer-reviewed venues in software engineering (IEEE TSE, ACM journals/conferences such as ICSE, ICSM, ASE, EASE, MSR), and journals such as *Information and Software Technology*, *Journal of Systems and Software*, *Empirical Software Engineering*, and *Expert Systems with Applications*.
- **Keyword clusters implied by the citation set:** “code smell detection,” “software metrics,” “code embedding,” “deep learning source code,” combined conceptually with specific smell names (God Class, Data Class, Feature Envy, Long Method) and specific model families (CodeBERT, CuBERT, code2vec).
- **Inclusion pattern:** studies directly addressing code smell/anti-pattern/design-defect detection, benchmark dataset papers (notably MLCQ; Madeyski & Lewowski, 2020), and pre-trained code representation models used as embedding backbones.
- **Exclusion pattern:** the paper does not cite general NLP survey literature outside of the specific models it adopts (BERT, RoBERTa, CodeBERT), and it restricts empirical

comparison to a narrow set of four smell types and one benchmark dataset (MLCQ) rather than surveying all available code-smell datasets.

Because this is a re-derived review rather than an independent search, the “database/inclusion-exclusion” transparency typical of a systematic review is limited to what is traceable through EnseSmells’ own bibliography.

11.3 Body Paragraphs (Thematic Synthesis)

11.3.1 Metric- and rule/heuristic-based detection

The earliest and most persistent thread in this literature treats code smells as violations of thresholds on object-oriented metrics. Marinescu (2004, 2005) proposed detection strategies using metrics like Weighted Method Count (WMC), Tight Class Cohesion (TCC), and Access To Foreign Data (ATFD), later formalized into commercial tooling (InCode; Marinescu et al., 2010). Lanza and Marinescu (2006) generalized this into logically-combined metric-threshold rules covering 11 anti-patterns. Sales et al. (2013) took a dependency-similarity approach (JMove) specifically for Feature Envy. Other tools in this vein include CCFinder (Kamiya et al., 2002), SpIRIT (Vidal et al., 2015), and DECOR (Moha et al., 2009).

Consensus and limitation: these studies agree that object-oriented metrics correlate meaningfully with several smell types, but they converge on a shared weakness - manually setting reliable thresholds is brittle and labor-intensive (Liu et al., 2016), and identical metric profiles can correspond to different underlying smells, producing classification ambiguity (Sharma et al., 2021).

11.3.2 Classical machine learning approaches

To avoid hand-tuned thresholds, a second wave of work applied supervised ML directly to metric vectors. Maiga et al. (2012a, 2012b) used SVMs (SVMDetect/SMURF) on ~60 structural metrics to detect God Class, Functional Decomposition, Spaghetti Code, and Swiss Army Knife. Khomh et al. (2011) applied Naive Bayes (BDTEX); Fontana and Zanoni (2017) used regression for severity classification; Arcelli Fontana et al. (2016) benchmarked 16 algorithms (and boosted variants) across four smells (Data Class, God Class, Feature Envy, Long Method) at multiple granularities, showing ML’s advantage over static rules.

Consensus and limitation: classical ML models learn the metric–label relationship rather than requiring hand-set thresholds, and generally outperform pure heuristics. However, they still depend entirely on hand-crafted metrics and largely ignore the *textual/semantic* content of source code, limiting their ability to disambiguate smells that share similar metric profiles.

11.3.3 Deep learning on source-code text

A third thread reframes source code as a signal amenable to deep neural representation learning, treating it initially much like natural-language text. This includes DNNs (Hadj-Kacem and Bouassida, 2018; Liu et al., 2021; Ho et al., 2022), CNNs (Liu et al., 2021; Das et al., 2019), LSTMs (Ho et al., 2023 - the authors’ own prior work, DeepSmells), and RNNs (Sharma et al., 2021, using autoencoder variants: AE-Dense, AE-CNN, AE-LSTM). These works show that deep architectures can extract useful patterns from raw token sequences without manual feature engineering.

Key finding and limitation: the paper explicitly identifies a shared shortcoming here - source code has syntactic and semantic structure fundamentally different from natural language (for example., nesting, control/data flow), and treating it as flat text risks losing information tied to those structural properties, which are often exactly what characterizes a given smell (for example., long/complex methods, high coupling).

11.3.4 Pre-trained code/language embedding models

A more recent trend leverages large pre-trained models specifically for code: code2vec/code2seq (Alon et al., 2019; Compton et al., 2020), CodeBERT (Feng et al., 2020, built on BERT/Transformer architectures - Devlin et al., 2019; Liu et al., 2019 RoBERTa), and CuBERT (Kanade et al., 2020). Kovačević et al. (2022) directly compared code2vec, code2seq, and CuBERT embeddings against traditional metrics for God Class and Long Method detection (their ML.CuBERT model is one of EnseSmells' primary baselines). Ma et al. (2024) further show that different pre-trained code models encode syntactic vs. semantic properties unevenly - a finding EnseSmells explicitly cites to justify its "statistical semantics" terminology (borrowed conceptually from Sikström and Garcia's *Statistical Semantics*, 2020, and adapted from natural-language distributional semantics to source code).

Key finding: these embedding-based approaches capture nuanced code semantics beyond token indexing but are rarely, in prior work, integrated *with* structural/metric information in a unified architecture - this is precisely the gap EnseSmells targets.

11.3.5 Methodological approaches within the reviewed literature

Across these strands, methodology has shifted from static rule engines → supervised classical ML on engineered features → deep sequence models on raw/tokenized code → embedding-based deep models - with a general (but not universal) trend toward improved detection accuracy, at the cost of interpretability and, in several deep-learning studies, robustness to severe class imbalance (a recurring problem the autoencoder baselines of Sharma et al., 2021 illustrate starkly with very high recall but very low precision).

11.4 Critical Evaluation

Gaps in research (as identified by EnseSmells and the surrounding literature):

- Prior deep-learning work overwhelmingly treats source code as text, under-weighting design-related/structural features (Ho et al., 2022; Liu et al., 2021; Sharma et al., 2021).
- Few studies systematically compare multiple pre-trained code embeddings (token indexing, code2vec, CodeBERT, CuBERT) *per smell type*, rather than assuming one embedding generalizes across all smells.
- Metric-based ML studies (Maiga et al., 2012a/b; Arcelli Fontana et al., 2016) rarely integrate learned semantic representations, leaving potential complementary information on the table.

Contradictions/debates: There is tension between metric-based and embedding-based camps regarding which signal is more predictive. Kovačević et al. (2022), for instance, argue embeddings can rival or outperform metrics for God Class/Long Method, while EnseSmells' own RQ1 results show metrics contribute substantial (up to ~40%) performance gains when added

to embeddings - suggesting the debate is smell-type-dependent rather than resolved in either direction.

Bias/limitations across the literature:

- Many baseline studies (autoencoder variants, ML_CuBERT) are evaluated on different datasets/splits than EnseSmells, so comparisons - even when re-run under the same conditions as the authors attempt - carry residual construct-validity risk.
- The MLCQ dataset itself (Madeyski and Lewowski, 2020) relies on majority-vote human labeling across only 26 reviewers, which the authors themselves flag as a threat to internal validity - human-labeled smell severity is inherently subjective.
- Severe class imbalance (for example., Feature Envy: 64 positive vs. 2,126 negative instances) affects nearly all cited empirical studies, and different works handle it inconsistently (weighted loss, oversampling, or not at all), complicating cross-study comparability.
- Generalizability is constrained: most cited empirical work, including EnseSmells, targets Java code and only 3–4 smell types, so conclusions may not transfer to other languages or architectural-level smells.

11.5 Conclusion and Future Directions

Summary. The literature on code smell detection has evolved from manually-thresholded metric rules, through classical ML classifiers on hand-crafted metrics, to deep learning models operating on source-code text, and finally to pre-trained code embedding models (code2vec, CodeBERT, CuBERT). Each stage improved automation and reduced reliance on manual threshold-setting, but largely treated structural/metric information and learned semantic representations as separate, competing signals rather than complementary ones.

Future scope. EnseSmells' contribution - and the natural next step this review points to - is the explicit fusion of statistical semantic features (via DeepSmells' CNN-LSTM pipeline over code embeddings) with structural features (via a DNN adaptive layer over CK-tool metrics), combined through an imbalance-aware classifier. Building on this, future work could: (1) extend the fusion approach to additional and architectural-level smells beyond the four studied here; (2) test generalizability across languages beyond Java and datasets beyond MLCQ; (3) investigate more principled, per-smell-type embedding selection (as RQ2's findings suggest different embeddings suit different smells) rather than a one-size-fits-all encoder; and (4) address class imbalance and label-subjectivity issues more directly, given how much these affect baseline comparability across the reviewed literature.

12 Human-In-the-Loop Software Development Agents

12.1 Introduction

Large Language Models (LLMs) have enabled a new class of autonomous and semi-autonomous *software engineering agents* capable of interpreting natural-language issue descriptions, localizing relevant source files, generating coding plans, and producing code changes with minimal human intervention. These systems are typically organized as *multi-agent* pipelines, in which distinct LLM-driven components are assigned specialized roles (for example., planner, coder, tester) that interact with external tools such as compilers, linters, and version-control systems to resolve software development tasks such as bug fixes, feature requests, and refactoring. A central design question in this space is the degree to which such agents should act autonomously versus incorporate *human-in-the-loop (HITL)* feedback, in which a software engineer reviews, edits, and approves agent-generated artifacts (plans, file selections, code) at each stage of the workflow.

The paper reviewed here, *Human-In-the-Loop Software Development Agents* (Takerngsaksiri et al.), introduces **HULA**, a three-agent framework (AI Planner Agent, AI Coding Agent, Human Agent) deployed inside Atlassian JIRA, and evaluates it through a multi-stage, real-world study.

12.1.1 Objective

This review synthesizes the literature and empirical evidence underlying the HULA paper in order to (1) situate HULA within the broader landscape of LLM-based software engineering agents, (2) examine the methodological approach used to evaluate such agents in an industrial setting, (3) critically assess the reported findings, gaps, and limitations, and (4) outline future research directions implied by the study.

12.1.2 Relevance

Most prior LLM-based software agent research is validated on static, open-source benchmarks (for example., SWE-bench) without human oversight and without real deployment, leaving open questions about generalizability to enterprise contexts, practitioner acceptance, and the practical role of human feedback. Understanding how a human-in-the-loop agent performs in a live, industrial software development pipeline—used by thousands of engineers—has direct implications for the design of trustworthy, human-centered AI development tools and for future empirical software engineering research.

12.2 Search Strategy (Methodology)

Because this review is anchored on a single primary paper, the “search strategy” reflects the related-work scope established by the authors themselves (Section III of the paper) rather than an independent multi-database search.

- **Source paper:** Takerngsaksiri et al., *Human-In-the-Loop Software Development Agents* (arXiv:2411.12924v2 [cs.SE]).
- **Cited literature scope:** The paper’s related-work section draws on peer-reviewed venues (ICSE, ESEC/FSE, ISSTA) and arXiv preprints covering LLM-based agents, bug localization, code completion, code review automation, and vulnerability detection/repair.

- **Keywords implicit in the cited literature:** “LLM-based agents,” “software engineering agents,” “issue resolution,” “code generation,” “bug localization,” “human-AI collaboration,” “SWE-bench.”
- **Inclusion criteria:** Studies proposing autonomous or semi-autonomous LLM-based agents for software development tasks (2021–2024); empirical studies on human-AI synergy in software engineering; benchmark datasets used for agent evaluation (SWE-bench, SWE-bench Verified).
- **Exclusion criteria:** General LLM survey papers not specific to software engineering were referenced only for background context; papers on narrow, single-task automation (for example., code completion alone) are treated as adjacent rather than core comparators, since HULA targets end-to-end issue resolution.

12.3 Body: Thematic Synthesis

12.3.1 Key Themes

The literature cited in the paper converges on a shared paradigm: an LLM agent receives environmental inputs (issue text, repository state, tool feedback) and takes actions (searching, editing, executing) toward a goal such as resolving a GitHub issue or pull request. Where the literature diverges is on **autonomy versus human involvement**. Existing agents (SWE-agent, AutoCodeRover, RepoUnderstander, Magis, CodeR, Masai) are designed to operate largely autonomously, optimizing for benchmark performance on historical, closed datasets, with no mechanism for iterative human feedback during plan or code generation. HULA departs from this consensus by explicitly foregrounding human authority at every stage (file selection, plan approval, code review), aligning with a smaller body of work on “trustworthy and synergistic AI for software engineering” and “augmented agile” human-centered AI-assisted management.

12.3.2 Methodological Approaches

- **Prior agent architectures:** SWE-agent uses a Linux-shell interface for search/navigate/edit/execute actions AutoCodeRover combines AST-based code search with spectrum-based fault localization; RepoUnderstander applies Monte Carlo tree search over a repository knowledge graph; Magis assigns four specialized roles (manager, custodian, developer, QA); CodeR uses predefined task graphs; Masai decomposes problems into modular sub-problems. These are largely evaluated via automated pass/fail metrics on SWE-bench-style benchmarks, with no human-in-the-loop stage.
- **HULA’s approach:** In contrast, HULA employs a Decentralized Planning Decentralized Execution (DPDE) coordination style—each agent (Planner, Coder, Human) works independently but shares memory (the issue and repository), minimizing inter-agent communication overhead while allowing human correction at each of four stages: task setup, planning, coding, and PR raising. Evaluation is multi-stage: (1) offline evaluation on SWE-bench Verified and an internal 369-issue JIRA dataset without human feedback; (2) online evaluation on 663 real-world JIRA issues with human feedback, tracked via a Sankey-style funnel of plan generation, approval, code generation, PR raising, and merging; (3) a 109-respondent practitioner survey (75% response rate) using Likert-scale and open-ended questions, analyzed via open coding and card sorting (Cohen’s $\kappa = 0.67$).

- **Limitation of prior methodology:** Benchmark-only evaluation (as used by comparator agents) cannot capture practitioner acceptance, human correction behavior, or deployment-scale reliability—goals that motivate HULA’s three-stage design.

12.3.3 Key Findings

- **Offline (RQ1):** On SWE-bench Verified, HULA achieves a 97% success-generation rate, 86% average file-localization recall, 84% perfect file localization, 45% average code-similarity score, and 31% perfect test-passing rate—comparable to SWE-agent Claude (6th on the SWE-bench leaderboard as of Sept. 2024). On the internal Atlassian dataset, performance drops substantially (100% success generation, but only 30% recall, 15% perfect localization, 30% high code similarity), attributed to greater language/repository diversity (10+ languages, 94 repositories) and shorter, less detailed issue descriptions (median 75 vs. 295 tokens).
- **Online (RQ2):** Of 663 real issues, 79% reached plan generation, 82% of generated plans were approved, 87% of approved plans led to code generation, 25% of generated code was raised as a PR, and 59% of raised PRs were merged—yielding an overall 8% end-to-end merge rate.
- **Survey (RQ3):** 71%/62% of practitioners agreed identified files were relevant/aligned with their own approach; only 41%/36% agreed generated plans matched the issue description/their own approach; 61% found generated code easy to read and modify, but only 33% agreed it solved the task without human involvement, and 54–67% disagreed that the code was defect-free or fully complete. Reported benefits: reduced development time/effort, help with simple tasks, encouragement of better documentation. Reported challenges: incorrect/incomplete code functionality, high effort in writing detailed issue descriptions, and rigid workflow (inability to revisit earlier stages).

12.4 Critical Evaluation

12.4.1 Gaps in Research

Prior LLM-agent literature (SWE-agent, AutoCodeRover, Magis, CodeR, Masai, RepoUnderstander) leaves three gaps that HULA explicitly targets: (1) reliance on historical, open-source benchmark datasets rather than industrial data; (2) absence of human feedback loops during generation; (3) lack of real-world deployment and practitioner perception studies. While HULA addresses all three, it in turn surfaces new gaps: the paper acknowledges that *little is known about what specific information* (for example., module names, code snippets, design patterns) should be included in an issue description to maximize agent performance, and there is no established methodology for automatically augmenting under-specified issues.

12.4.2 Contradictions

There is a tension between the offline and survey results: SWE-bench-style automated metrics (for example., 45% code similarity, 31% test-pass rate) suggest moderate objective performance, yet practitioners still rate the plan-issue alignment low (41%) and code completeness low (10% strongly agree), indicating that automated similarity/pass-rate metrics may not fully capture practitioner-perceived quality—a contradiction the authors themselves flag as “functional correctness should go beyond passing unit test cases.” Likewise, the strong recall for

file localization on SWE-bench (86%) versus internal data (30%) contradicts the assumption, common in prior benchmark-only studies, that agent competence generalizes across codebases and languages.

12.4.3 Bias / Limitations

- **Construct validity:** Code similarity is measured via GPT-4-as-a-judge rather than direct human annotation for the full dataset (validated on only a 203-issue subsample with $r = 0.7$ correlation to human judgment)—introducing potential automation bias.
- **External validity:** Findings are confined to Atlassian’s internal JIRA/Bitbucket environment and a GPT-4 backbone; generalizability to other organizations, codebases, or LLMs is untested.
- **Internal validity:** The study does not compare against alternative backbone LLMs, so it is unclear whether reported gains stem from the HULA framework itself or from GPT-4’s specific capabilities.
- **Selection/response bias:** The practitioner survey, while achieving a 75% response rate, only sampled engineers who had already completed the HULA workflow at least once, potentially excluding early dropouts with more negative experiences.

12.5 Conclusion and Future Directions

12.5.1 Summary

The HULA paper contributes a deployed, human-in-the-loop, multi-agent framework for software development integrated into a real enterprise tool (JIRA), evaluated through offline benchmarking, live deployment metrics, and practitioner surveys. It demonstrates that human feedback substantially improves practical acceptance (82% plan approval, 59% merged-PR rate among raised PRs) even though fully autonomous code generation still falls short of practitioner expectations for completeness and defect-freedom.

12.5.2 Future Scope

Building on the identified gaps, future research (as also proposed by the authors) should investigate: (1) what categories of contextual information (dependencies, design patterns, historical solutions) most improve LLM agent performance, and how to automatically augment issue descriptions with this information; (2) evaluation metrics for functional correctness that go beyond unit-test pass rates, incorporating readability, maintainability, and technical debt; (3) more flexible, non-linear HITL workflows that let engineers revisit earlier stages (planning after coding) rather than following a strictly linear pipeline; (4) comparative studies across different backbone LLMs and across other organizations/platforms to test the generalizability of the HULA framework.

13 LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead

13.1 Introduction

Autonomous agents-computational entities capable of perceiving their environment, planning strategically, and executing actions-have been studied since the 1990s, but early implementations lacked human-like reasoning. The emergence of Large Language Models (LLMs) such as GPT-4, Claude, and Gemini has changed this landscape by endowing agents with near-human planning and reasoning capabilities. Because most real-world software engineering (SE) problems span multiple domains and require diverse expertise, researchers have increasingly turned to **LLM-based Multi-Agent (LMA) systems**, in which several specialized LLM-driven agents collaborate-through debate, division of labor, and cross-validation-to solve tasks that a single agent cannot handle alone. An LMA system, as defined by He et al. (2025), consists of two core components: an *orchestration platform* (governing coordination models, communication mechanisms, and planning/learning styles) and a set of *LLM-based agents* (which may be predefined or dynamically generated, homogeneous or heterogeneous).

13.1.1 Objective

This review synthesizes the state of research on LMA systems applied across the software development lifecycle (SDLC)-requirements engineering, code generation, quality assurance, and maintenance-as well as end-to-end software development. It further examines the paper's empirical case studies of a state-of-the-art LMA framework (ChatDev) and distills the research agenda the authors propose for advancing the field.

13.1.2 Relevance

Software systems are growing rapidly in scale and complexity, straining traditional development practices and even single-agent LLM approaches, which remain prone to hallucination and single-point failure. LMA systems promise autonomous problem-solving, improved robustness through cross-examination (analogous to code review), and scalability through distributed, specialized agents. Understanding the current landscape, its limitations, and open research questions is essential for guiding the transition toward what the authors term “Software Engineering 2.0.”

13.2 Search Strategy (Methodology)

13.2.1 Database

The primary source was the **DBLP Computer Science Bibliography**, a widely used database in SE surveys, indexing over 7.5 million publications across 1,800 journals and 6,700 conferences.

13.2.2 Keywords

Two keyword sets were combined with the Boolean AND operator:

- **Agent-related terms:** “Agent” OR “LLM” OR “Large Language Model” OR “Collaborat”

- **SE-phase-specific terms:**
 - Requirements Engineering: *requirement, specification, stakeholder*
 - Code Generation: *software, code, coding, program*
 - Quality Assurance: *bug, fault, defect, fuzz, test, vulnerab, verificat, validat*
 - Maintenance: *debug, repair, review, refactor, patch, maintenanc*

Truncated stems (for example., “vulnerab”) were used to capture morphological variants.

13.2.3 Inclusion and Exclusion Criteria

Inclusion:

- Written in English with accessible full text.
- Adopts LMA techniques (multiple collaborating LLM-based agents) to solve SE tasks.

Exclusion:

- Fewer than 5 pages.
- Duplicate or highly similar studies by the same authors.
- Books, keynotes, panel summaries, technical reports, theses, tool demos, editorials.
- Literature reviews or surveys.
- Studies using only a single LLM agent (non-agentic or single-agent).
- Published before November 2022 (pre-ChatGPT release).
- Not SE-related.
- LMA systems mentioned only as future work, without experimental results.

Screening proceeded in three phases (removal of short/duplicate papers, manual title/abstract screening, full-text review), yielding **41 primary studies**. **Backward and forward snowballing** was then applied iteratively until reaching a fixed point, adding **30 more papers**, for a total of **71 primary studies** (search conducted November 14, 2024).

13.3 Body: Thematic Synthesis

13.3.1 Requirements Engineering

LMA systems have been applied to elicitation, modeling, specification, and validation. **Elicitron** simulates diverse users via LLM agents to surface user needs. **MARE** uses five role-specialized agents (stakeholder, collector, modeler, checker, documenter) across multiple RE phases. Sami et al. propose a four-agent framework (product owner, developer, QA, manager) for generating and prioritizing user stories collaboratively.

13.3.2 Code Generation

A recurring architectural pattern emerges: **Orchestrator, Programmer, Reviewer, Tester, and Information Retriever** roles. Frameworks such as PairCoder (Navigator/Driver), Self-Organized Agents (Mother/Child hierarchy), and CODES (RepoSketcher/FileSketcher/Sketch-Filler) illustrate hierarchical task decomposition. Iterative feedback loops (for example., INTERVENOR’s Code Learner/Code Teacher pairing) support debugging, while retrieval agents (Agent4PLC, MapCoder, CodexGraph) supply contextual knowledge. Agent Forest departs from role specialization, instead using sampling-and-voting across independently generated candidate solutions.

13.3.3 Software Quality Assurance

Sub-themes include:

- **Testing:** Fuzz4All, AXNav (accessibility testing), WhiteFox (compiler fuzzing).
- **Vulnerability Detection:** GPTLens (auditor + critic agents for smart contracts), MuCoLD, and cross-validation techniques (Widyasari et al.).
- **Bug Detection:** ICAA, combining report generation with false-positive pruning.
- **Fault Localization:** RCAgent (cloud root-cause analysis), AgentFL (Comprehension/Navigation/Confirmation agents).

13.3.4 Software Maintenance

Debugging frameworks (MASAI, MarsCode, AutoSD, FixAgent, MASTER, AutoCodeOver, SpecRover) generally decompose the task into bug reproduction, fault localization, patch generation, and validation, often using specification analysis or Monte Carlo tree search (for example., SWE-Search, RepoUnderstander). Code-review-focused systems (Rasheed et al., CodeAgent) add supervisory “QA-Checker” agents to maintain coherence among reviewing agents.

13.3.5 End-to-End Software Development

Full-lifecycle systems typically mirror established process models:

- **Waterfall-inspired:** MetaGPT (Product Manager → Architect → Engineer → QA Engineer), and related systems.
- **Agile-inspired:** AgileCoder, AgileGen (sprint-based roles, Gherkin-based testable requirements).
- **Dynamic process generation:** Think-on-Process and MegaAgent generate roles/workflows on the fly rather than relying on fixed pipelines.
- **Experience-driven:** Co-Learning and iterative experience refinement (IER) leverage historical project data to improve future collaboration.

13.3.6 Key Findings from Case Studies

Using ChatDev (GPT-3.5-turbo backend) to generate a Snake game and a Tetris game, the authors found LMA systems handle *moderately complex* tasks efficiently (Snake: succeeded on the 2nd attempt, \$0.019/76s) but struggle with tasks requiring deeper logical abstraction (Tetris: succeeded only on the 10th attempt, and still lacked row-clearing logic).

13.4 Critical Evaluation

13.4.1 Gaps in Research

- **Role specialization limits:** current agents rely on general-purpose LLMs that lack nuanced SE domain expertise (for example., security auditing, vulnerability repair).
- **Prompting languages remain human-centric:** existing frameworks (DSPy, AutoGen, LangChain) prioritize developer readability rather than LLM-native comprehension.
- **Lack of collaboration-aware benchmarks:** existing benchmarks (for example., LiveCodeBench, BigCodeBench) evaluate isolated coding tasks, not multi-agent negotiation, conflict resolution, or peer review.

- **Scalability:** static architectures with fixed roles and communication patterns struggle as project size grows; no existing work fully supports on-the-fly agent/role reconfiguration.
- **Privacy and cross-organizational data sharing:** unresolved for multi-organizational SE contexts (no LMA-specific solution yet exists, despite general security-literature techniques such as differential privacy or SMPC).

13.4.2 Contradictions and Debates

The paper contrasts LMA systems with the **Mixture-of-Experts (MoE)** paradigm, arguing LMA is preferable for SE because MoE experts do not communicate or incorporate external tool feedback, whereas LMA agents actively collaborate and can integrate compilers/static analyzers. This is a normative claim rather than an empirically settled comparison, and no head-to-head evaluation is provided.

13.4.3 Bias/Limitations

- The review relies primarily on a single database (DBLP) plus snowballing; despite mitigation efforts, some relevant preprints or venue types may be missed (acknowledged by the authors as a threat to validity).
- Case studies are limited to two toy tasks (Snake, Tetris) and a single framework (ChatDev) with a single backend LLM (GPT-3.5-turbo), limiting generalizability of the “current capability” claims.
- The proposed research agenda is largely prescriptive/visionary rather than derived from quantitative gap analysis across the 71 studies.

13.5 Conclusion and Future Directions

13.5.1 Summary

The reviewed literature shows LMA systems have been applied across nearly every SDLC phase, most maturely in code generation and quality assurance, with role specialization (Orchestrator, Programmer, Reviewer, Tester) as the dominant design pattern. End-to-end systems largely imitate Waterfall or Agile processes, with a smaller but growing trend toward dynamic, experience-driven architectures. Case studies confirm current systems are efficient and cost-effective for moderately complex tasks but falter on tasks demanding deep logical reasoning.

13.5.2 Future Scope

The authors propose a two-phase research agenda:

1. **Phase 1 - Enhancing Individual Agent Capabilities:** refining role-playing through market analysis, competency mapping, and targeted fine-tuning; developing agent-oriented (rather than human-oriented) prompting languages.
2. **Phase 2 - Optimizing Agent Synergy:** improving human-agent collaboration protocols, building collaboration-aware benchmarks, scaling to complex projects via hierarchical task decomposition, adopting industrial process frameworks (DDD, BDD, Team Topologies), enabling dynamic runtime adaptation, and ensuring privacy-preserving data management (RBAC/ABAC extensions, differential privacy, federated learning).

These directions collectively aim toward autonomous, scalable, trustworthy LMA systems-what the authors term “Software Engineering 2.0.”

14 A Causal Perspective on Measuring, Explaining and Mitigating Smells in LLM-Generated Code

14.1 Introduction

Large language models (LLMs) have become central to modern software engineering (SE) workflows, powering tasks such as code completion, summarization, program repair, clone detection, and test generation. As LLM-generated code increasingly enters production systems, concerns have grown about its *structural quality*—not merely whether the code runs correctly, but whether it exhibits *code smells*: recurring design and implementation patterns that hinder readability, maintainability, and long-term robustness. Traditionally, code smells were understood as artifacts of human factors such as developer experience, time pressure, and project constraints. However, LLMs, trained on large-scale code corpora, inherit smelly or suboptimal patterns from their training data and may introduce additional defects during generation, complicating this traditional attribution.

14.1.1 Objective

This review examines the literature underpinning Velasco et al.’s (2026) study, “A *Causal Perspective on Measuring, Explaining and Mitigating Smells in LLM-Generated Code*.” The paper builds on the previously introduced Propensity Smelly Score (PSC)—a probabilistic metric estimating the likelihood that an LLM generates a given code smell—and extends it through (1) robustness and information-gain validation, (2) causal analysis of the generation pipeline (decoding strategy, model size, architecture, and prompt formulation), (3) prompt-based mitigation strategies, and (4) a human-subject study of PSC’s practical utility. The objective of this review is to situate these contributions within the broader literature on LLM code quality evaluation, causal inference methods, and human-centered assessment of AI-assisted development tools.

14.1.2 Relevance

As LLM-based coding assistants (for example., GitHub Copilot, ChatGPT, CodeWhisperer) become embedded in everyday development, evaluation paradigms that focus solely on functional correctness (for example., pass@k, unit test success) or lexical similarity (for example., BLEU, CodeBLEU) are insufficient to characterize the long-term maintainability risks of AI-generated code. Understanding *why* and *when* LLMs introduce smells—and whether such tendencies can be causally attributed to specific generation-pipeline components—has direct implications for responsible AI deployment, developer trust, and the design of quality-aware evaluation benchmarks for code-generation models.

14.2 Search Strategy (Methodology)

14.2.1 Databases

The literature synthesized in this review is drawn primarily from the reference list of the target paper, supplemented conceptually by adjacent work in software engineering venues (ICSE, ASE, MSR, ISSTA) and preprint repositories (arXiv cs.SE, cs.CL, cs.CR). Representative databases relevant to this domain include *arXiv*, *ACM Digital Library*, *IEEE Xplore*, and *Google Scholar*.

14.2.2 Keywords

Search terms informing the underlying body of work include combinations such as: "code smells" AND "large language models"; "LLM-generated code" AND "quality evaluation"; "causal inference" AND "software engineering"; "prompt engineering" AND "code quality"; "BLEU" OR "CodeBLEU" AND "code generation evaluation"; and "test smells" AND "LLM".

14.2.3 Criteria

Sources considered are predominantly peer-reviewed or archival preprints published between 2015 and 2026, with the majority concentrated in 2022–2025, reflecting the rapid emergence of LLM-for-code research. Inclusion criteria emphasized empirical studies on code smell detection/generation in LLM outputs, causal or explanatory methods applied to code generation, and evaluation-metric papers (BLEU, CodeBLEU, PSC). Foundational but older works (for example., Tufano et al. 2015 on human-introduced smells; Papineni et al. 2002 on BLEU) were included for historical grounding despite falling outside the recent window.

14.3 Body: Thematic Synthesis

14.3.1 Theme 1: Limitations of Correctness-Centric Evaluation

A substantial line of work establishes that functional-correctness metrics are inadequate proxies for code quality. Evtikhiev et al. (2023) critique BLEU and similar n-gram metrics for their inability to capture semantic or maintainability differences between generated and reference code. Ren et al. (2020) proposed CodeBLEU to incorporate syntactic and data-flow information, yet this metric remains fundamentally reference-based and continues to prioritize surface-level similarity over structural design quality. This theme establishes the motivation for PSC: a metric grounded not in similarity to a reference, but in the model's own token-generation probabilities.

14.3.2 Theme 2: Empirical Evidence of Smells in LLM-Generated Code

Multiple empirical studies document that LLMs reproduce or introduce code smells. Siddiq et al. (2022) and Melin et al. (2024) show that both training corpora and generated outputs of transformer-based models frequently contain code smells and insecure constructs. Mohsin et al. (2024) found that general-purpose conversational models (Bard, ChatGPT) introduce more smells than specialized coding assistants (Copilot, CodeWhisperer) on complex tasks. Kharmah et al. (2025) linked the persistence of legacy antipatterns to outdated training data. In the test-generation domain specifically, Ouédraogo et al. (2024) reported high rates of test smells in GPT-3.5/GPT-4 outputs. Collectively, these studies establish the empirical premise motivating causal investigation: smell generation is a real, widespread, and model-dependent phenomenon.

14.3.3 Theme 3: Emerging Benchmarks and Metrics for Structural Quality

A more recent cluster of work moves beyond detection toward structured evaluation. Velasco et al. (2025), in the CodeSmellEval benchmark, introduced the PSC metric itself. Zheng et al. (2024) proposed the RACE framework to benchmark multiple quality dimensions (correctness, readability, maintainability) simultaneously. Wu et al. (2024) introduced iSMELL, combining static analysis tools with LLMs for detection and refactoring. These works share a common limitation: they measure or detect smells post hoc but do not explain *why* a model produces

them or which generation-pipeline components are responsible—precisely the gap the reviewed paper addresses.

14.3.4 Theme 4: Causal Inference Applied to Code Generation

The paper’s methodological backbone draws on causal inference theory (Pearl & Mackenzie, 2018) and its software-engineering applications. Rodriguez-Cardenas et al. (2023) previously benchmarked causal methods for interpreting LLMs on source code, establishing the structural-causal-model (SCM) approach later adopted here. Sharma and Kiciman’s DoWhy library (2020) provides the computational infrastructure for average-treatment-effect (ATE) estimation and refutation testing (placebo tests, random common causes, subset validation), which the paper employs to establish the robustness of its causal claims regarding decoding strategy, model size, architecture, and prompt design.

14.3.5 Theme 5: Generation-Pipeline Factors Influencing Output Structure

Prior work independently supports the paper’s choice of confounders and treatment variables. Donato et al. (2025) show that sampling temperature affects code length and nesting depth; Chen et al. (2021) (Codex) note that larger models tend to produce more concise completions; Jiang et al. (2023) demonstrate that architectural choices (for example., Mistral’s grouped-query attention and sliding-window design) yield systematically different output lengths than comparably sized LLaMA-family models; and Khojah et al. (2024) show that prompt specificity shapes completion length and structure. These findings are synthesized into the paper’s four causal treatments (generation strategy, model size, architecture, prompt type).

14.3.6 Theme 6: Human-Centered Evaluation of AI-Assisted Code Review

The paper’s user study connects to a smaller but growing literature on how developers interpret AI-generated artifacts. Baltes and Ralph’s (2021) guidelines on purposive sampling in SE research inform the study’s participant recruitment strategy, while Cruzes and Dyba’s (2011) thematic synthesis method underlies the qualitative analysis of open-ended survey responses. This theme reflects an evolving recognition that quantitative smell metrics alone are insufficient—their practical value must be validated against human judgment and confidence in real code-review contexts.

14.4 Critical Evaluation

14.4.1 Gaps in Research

Prior to this study, three gaps persisted in the literature: (1) the *robustness* of PSC under syntactic variation had not been tested, leaving open whether the metric reflects deep generative tendencies or superficial token patterns; (2) no study had systematically decomposed *which* generation-pipeline components (decoding, scale, architecture, prompting) causally drive smell propensity, as opposed to merely correlating with smell frequency; and (3) despite metric proposals (PSC, RACE, iSMELL), none had been operationalized into actionable, inference-time mitigation strategies or validated for practical usefulness with human developers.

14.4.2 Contradictions and Debates

A notable tension exists between the intuitive assumption—echoed in some prior literature (for example., scaling-law narratives around larger models producing “better” code)—and the paper’s finding that model size exerts only weak, statistically fragile effects on smell propensity. This contradicts a common assumption in the LLM-for-code literature that scaling model parameters alone improves output quality; the paper’s causal results instead attribute meaningful improvements to architecture and prompt design, not scale. Additionally, findings on decoding strategy are mixed: sampling-based methods reduce propensity for certain semantic smells (for example., unnecessary dunder calls) while increasing it for formatting-related smells (for example., trailing whitespace), suggesting decoding strategy is not a uniformly beneficial lever, in partial tension with prior work suggesting greedy decoding is categorically “safer.”

14.4.3 Bias and Limitations

Several methodological limitations merit scrutiny. First, the causal analysis relies on a confounder set (LOC, AST height, POS tag counts, etc.) that, while empirically motivated, omits design-level features such as class hierarchy, call-graph structure, and cross-file dependencies—these cannot be extracted from isolated snippets, introducing potential residual confounding. Second, the evaluation is confined to Python and decoder-only architectures; generalizability to other languages (for example., Java, C++) and architectures (encoder-decoder models) remains unverified, despite the authors’ claim of methodological extensibility. Third, the user study’s sample size (36 valid responses split across two conditions) is modest, and while purposive sampling ensured domain expertise, it limits statistical power and external validity of the human-subject findings. Finally, reliance on Pylint as the sole source of smell ground truth may introduce a systematic bias toward smells that this particular static analyzer is designed to detect, potentially underrepresenting other smell taxonomies.

14.5 Conclusion and Future Directions

14.5.1 Summary

The reviewed literature traces a clear trajectory: from early recognition that correctness-based metrics (BLEU, CodeBLEU) inadequately capture code quality, through empirical documentation of smell prevalence in LLM outputs, to the emergence of specialized metrics (PSC, RACE, iSMELL) for structural evaluation, and finally to the causal, explanatory framework advanced by the target paper. By integrating causal inference tooling (DoWhy) with a probabilistic smell metric and validating its behavior through robustness testing, information-gain comparison, and a controlled user study, the paper substantially extends prior detection-oriented work into an explanatory and mitigative paradigm.

14.5.2 Future Scope

Building on the identified gaps, future research should: (1) extend PSC and causal analysis to additional programming languages and encoder-decoder architectures to test generalizability; (2) incorporate richer design-level confounders (for example., inter-procedural and object-oriented structural features) to reduce residual confounding in causal estimates; (3) scale the user study to larger, more diverse developer populations to strengthen external validity of usefulness claims; and (4) develop comprehensive mitigation frameworks that move beyond single-prompt case

studies toward integrated, deployable inference-time quality controls for LLM-based coding assistants.

15 FEDCSD: A FEDERATED LEARNING BASED APPROACH FOR CODE-SMELL DETECTION

15.1 Introduction

Software quality assurance is a persistent concern in software engineering, with *code smells* - surface-level indicators of deeper design or implementation problems such as poor cohesion, excessive complexity, or tight coupling - serving as a key diagnostic construct. Although code smells do not cause compilation or runtime errors, their presence is strongly associated with reduced maintainability and increased *technical debt*. Among the many recognized smells, the *God Class* (or Large Class/Blob) - a class that accumulates too many responsibilities - has received particular attention due to its frequent occurrence and strong negative impact on maintainability.

Detection of code smells has evolved from manual inspection, to metric- and rule-based tools (for example., iPlasma, DECOR, PMD, JDeodorant), to machine learning (ML)-based classifiers trained on software metrics. More recently, *Federated Learning* (FL) - a distributed learning paradigm in which multiple parties collaboratively train a shared model without exchanging raw data - has been proposed as a privacy-preserving alternative to centralized ML training, particularly relevant when the training data (source code) is commercially sensitive.

15.1.1 Objective

This review synthesizes the literature discussed in on (i) traditional code smell detection techniques, (ii) ML-based detection approaches, and (iii) the emergence of Federated Learning as a privacy-preserving alternative for collaborative, cross-organizational code smell detection. The objective is to characterize how the reviewed work motivates and positions the FedCSD approach, and to critically evaluate the strength of the evidence base underpinning it.

15.1.2 Relevance

Understanding this research area matters for two intersecting reasons. First, code smell detection is directly tied to reducing long-term software maintenance costs. Second, as organizations become increasingly reluctant to expose proprietary source code for external analysis (due to GDPR and competitive concerns), techniques that preserve data privacy while still enabling collaborative model improvement - such as FL - represent a potentially significant shift in how software quality analytics can be conducted across organizational boundaries.

15.2 Search Strategy (Methodology)

Because this review is derived from a single source paper rather than an independent multi-database search, this section describes the scope and provenance of the literature as reported by the authors, rather than an original search protocol.

- **Source paper's approach:** The authors situate their work through a narrative and systematic-mapping-informed review (drawing especially on a prior mapping study of nearly 400 articles on code smells, covering detection-tool literature, ML-based detection literature, and FL literature).

- **Scope of works cited:** Studies span code smell detection tools (iPlasma, DECOR, PMD, JDeodorant), ML-based smell detection (decision trees, SVM, Bayesian classifiers, ensemble methods), and general FL literature (FedAvg, FEDn framework, FL security/privacy threats).
- **Implicit inclusion criteria:** Peer-reviewed venues (journals, conferences), spanning roughly 1993–2023, with emphasis on empirical studies that used open-source Java systems for God Class/Large Class detection.
- **Datasets referenced:** Three previously published/validated datasets are reused rather than newly collected, situating the empirical contribution within an existing benchmark landscape.

15.3 Body: Thematic Synthesis

15.3.1 Theme 1 - Rule- and Metric-Based Detection Tools

Early and still widely used approaches rely on static software metrics and rule thresholds (for example., iPlasma, jCosmo, InCode, DECOR, PMD, Borland Together, JDeodorant). These tools are reported to suffer from low inter-tool agreement, limited language support, poor interoperability across development environments, and doubtful scalability to large codebases. This theme establishes the baseline problem that ML-based approaches subsequently attempt to address: detection tools disagree with each other and with human experts.

15.3.2 Theme 2 - Machine Learning-Based Detection

A substantial body of work applies supervised ML to metrics-derived features for smell classification:

- Decision trees for Large Class/Long Method detection;
- Prediction of multiple design smells from metric sets;
- Bayesian approaches (BDTEX) for antipattern detection;
- SVM-based detection of Blob, Spaghetti Code, and related antipatterns;
- Large-scale empirical comparisons using 16–28 classifiers across dozens of systems;
- Comparative studies of metric-based vs. ML-based detection performance.

Collectively, these studies conclude that ML classifiers can achieve high detection accuracy, but performance is sensitive to dataset composition, class balance, and domain diversity of the training systems.

15.3.3 Theme 3 - Centralized ML's Structural Limitation

Across the reviewed studies, a consistent methodological pattern emerges: models are trained on data aggregated centrally from open-source repositories. The paper argues this creates two compounding problems: (1) centralization is costly and creates a single point of failure/attack surface; and (2) it is fundamentally incompatible with closed-source, proprietary industry codebases, since companies will not release source code for centralized training. This is presented as the key unaddressed gap that motivates the shift to FL.

15.3.4 Theme 4 - Federated Learning as a Privacy-Preserving Alternative

FL is introduced as an architecture in which model updates (weights/gradients), rather than raw data, are exchanged between clients and an aggregation server. The paper adopts the FEDn framework, structured hierarchically into reducers, combiners, and clients, using the FedAvg aggregation algorithm. Prior applications of FL to sensitive-data domains (like health monitoring) are cited as precedent, but the authors note - and this review confirms - that no prior study had applied FL specifically to code smell detection prior to this paper.

15.3.5 Theme 5 - Security and Privacy Threats in FL

A smaller but relevant strand of literature addresses adversarial risks in FL/ML pipelines generally: model poisoning, data poisoning, gradient inversion, and membership inference attacks, alongside general adversarial example literature. This theme is discussed conceptually but is explicitly *not* empirically tested within the paper itself.

15.3.6 Methodological Approaches Across the Literature

Most reviewed ML studies follow a common empirical template: extract object-oriented metrics from open-source Java systems, apply a classifier, and validate detection against tool output and/or manual expert labeling. Validation sample sizes range from single systems to over 70 systems. The source paper distinguishes itself methodologically by adopting Design Science Research and a GQM (Goal-Question-Metric) formulation to structure hypothesis-driven evaluation - a more formalized methodology than most of the antecedent ML studies, which are largely empirical/exploratory.

15.3.7 Key Findings Synthesized

- Rule-based tools have low mutual agreement and limited scalability.
- ML classifiers achieve high nominal accuracy but their performance is dataset-dependent, with datasets of lower smell diversity/coverage producing markedly lower accuracy (for example., 92.30% vs. 98.90–99.50% in the centralized experiments reported).
- Cross-dataset generalization is weak: models trained on one company/dataset degrade substantially (accuracy as low as 63.80%) when applied to differently-distributed data, evidencing a concept/data drift problem consistent with technical debt accumulation.
- FL-based aggregation across ten simulated companies achieves accuracy (98.34%) comparable to the best centralized single-dataset model, while, unlike centralized training, preserving data locality.

15.4 Critical Evaluation

15.4.1 Gaps in Research

- Prior to this paper, there was no identified application of FL specifically to code smell detection - a genuine and clearly substantiated gap.
- The broader ML-based smell detection literature has concentrated heavily on open-source Java systems; there is little empirical evidence about detection performance on closed-

source, proprietary, or non-Java codebases, a limitation the source paper inherits and explicitly acknowledges (External Validity, Section 7.3).

- Security/adversarial robustness of FL in this specific application domain (model poisoning, gradient inversion, membership inference) is discussed only conceptually, not empirically evaluated - a gap the authors themselves flag as future work.

15.4.2 Contradictions and Debates

- There is a tension between studies claiming metric-based approaches slightly outperform ML approaches and the broader literature (and this paper) treating ML/FL as the superior direction - suggesting the field has not fully reconciled which paradigm is preferable under which conditions.
- Reported detection agreement among human experts and tools is described elsewhere in the literature as low, yet many ML studies rely on tool-generated or expert-validated labels as ground truth, raising an internal tension about label reliability across the field.

15.4.3 Bias, Reliability, and Limitations

- **Construct validity:** Reliance on a single framework (FEDn) and a single algorithm family (LSTM) limits claims about generalizability to other FL frameworks or model architectures; the authors justify this by algorithm-availability constraints at the time of the study rather than a principled comparison.
- **Internal validity:** Class imbalance (God Class vs. Not-God-Class) is addressed via oversampling/undersampling, a common but imperfect mitigation that can itself introduce distributional bias.
- **External validity:** All three datasets consist of open-source systems; extrapolating results to closed-source industrial settings - the very use case motivating the FL approach - remains unverified.
- **Single-smell focus:** The literature synthesis and the paper's own contribution are restricted to God Class; conclusions may not transfer to other smell types (for example., Feature Envy, Long Method) without further validation.
- **Simulated federation:** The ten "companies" in Experiment 3 are simulated by partitioning existing centralized datasets rather than drawn from genuinely independent organizational data silos, which somewhat limits the ecological validity of the privacy-preservation claims.

15.5 Conclusion and Future Directions

15.5.1 Summary

The literature establishes a clear trajectory: from rule-based detection tools (limited by low agreement and scalability), to ML-based classifiers (higher accuracy but dataset- and domain-sensitive), to the still-nascent application of Federated Learning as a mechanism for reconciling high detection accuracy with data/code privacy across organizational boundaries. The reviewed paper's FedCSD approach is positioned as the first to close this specific gap for code smell

detection, demonstrating experimentally that federated training can approach centralized-model accuracy without requiring data centralization.

15.5.2 Future Scope

Future research building on this literature should address:

- Empirical evaluation of FL-based code smell detection on genuinely distributed, closed-source industrial datasets rather than partitioned open-source benchmarks;
- Extension to additional code smell types beyond God Class, and to non-Java languages;
- Empirical (not only conceptual) assessment of adversarial robustness - model poisoning, gradient inversion, and membership inference attacks - within FL-based code smell detection pipelines;
- Comparative evaluation of multiple FL frameworks and model architectures (beyond FEDn/LSTM) to strengthen construct validity claims.

16 Contribution

Student Id	Contribution
210042111	EnseSmells: Deep ensemble and programming language models for automated code smells detection
	Human-In-the-Loop Software Development Agents
	LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead
	A Causal Perspective on Measuring, Explaining and Mitigating Smells in LLM-Generated Code
	FEDCSD: A FEDERATED LEARNING BASED APPROACH FOR CODE-SMELL DETECTION
210042112	AI-powered Code Review with LLMs: Early Results
	A Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System
	Software Maintainability Index Prediction with Source Code Metrics using Machine Learning
	An Empirical Study on the Potential of LLMs in Automated Software Refactoring
	An Empirical Evaluation of Code Smell Detection in Angular Applications
210042114	Patterns of Developer Adoption of LLM-Generated Code Refactoring Suggestions
	A Systematic Literature Review of Agentic AI: Definitions, Architectures, and Challenges
	Exploring the Use of Large Language Models for Data Extraction for Systematic Reviews in Software Engineering
	CodeSense: A Static Analysis Tool for Detecting Code Smells
	Code Smell Detection Research Based on Pre-training and Stacking Models

Table 1: Contribution Table